



APOSTILA DA DISCIPLINA

Visão Computacional Aplicada

Processamento de Imagem, Reconhecimento de Padrões e Aplicações de IA — na prática, no navegador, em Python e em Node.js

Prof. Claudiney Sanches

FIAP — Faculdade de Informática e Administração Paulista

2026

Sumário

Apresentação	3
Como usar esta apostila	4
Seção 0 — Preparando o Ambiente de Desenvolvimento	5

C1	Capítulo 1 — Introdução à Visão Computacional e Inteligência Artificial Processamento de imagens x visão computacional, IA/ML/Deep Learning, aplicações reais e o stack do curso.
C2	Capítulo 2 — Fundamentos de Processamento de Imagens Pixels, canais de cor, espaços de cor (RGB, HSV, escala de cinza) e a estrutura de uma imagem digital.
C3	Capítulo 3 — Operações Básicas de Imagem Manipulação de pixels, operações aritméticas e lógicas entre imagens, brilho e contraste.
C4	Capítulo 4 — Histogramas, Limiarização e Binarização Histograma de intensidades, equalização, thresholding simples e adaptativo.
C5	Capítulo 5 — Segmentação: Crescimento de Região e Componentes Conexos Algoritmos de crescimento de semente, 4 e 8-conectividade, rotulação de componentes.
C6	Capítulo 6 — Filtros Espaciais e Convolução I Convolução, filtro da média, filtro gaussiano e suavização de ruído.
C7	Capítulo 7 — Filtros Espaciais II: Bordas e Contornos Filtro mediano, laplaciano, gradiente, Sobel e Canny.
C8	Capítulo 8 — Laboratório Integrador: Checkpoint Prático Consolidação das aulas 1 a 7 em um mini-projeto guiado.
C9	Capítulo 9 — Casamento de Template I Correlação cruzada, normalização e o problema do casamento de padrões.

- | | |
|------------|--|
| C10 | Capítulo 10 — Casamento de Template II
Aplicações, limitações e invariância a escala/rotação. |
| <hr/> | |
| C11 | Capítulo 11 — Transformações Geométricas I
Mudança de escala e interpolação (vizinho mais próximo, bilinear). |
| <hr/> | |
| C12 | Capítulo 12 — Transformações Geométricas II
Rotação, translação, transformação afim e de perspectiva. |
| <hr/> | |
| C13 | Capítulo 13 — Morfologia Matemática
Erosão, dilatação, abertura, fechamento e elemento estruturante. |
| <hr/> | |
| C14 | Capítulo 14 — Introdução ao Aprendizado de Máquina em VC
Aprendizado supervisionado, extração de atributos e classificadores clássicos. |
| <hr/> | |
| C15 | Capítulo 15 — Redes Neurais e CNNs Aplicadas à VC
De MLP a CNN, convolução como extrator automático de atributos. |
| <hr/> | |
| C16 | Capítulo 16 — Aplicações de IA: Detecção, Reconhecimento e OCR
Detecção de objetos, reconhecimento facial e leitura de texto em imagens. |
| <hr/> | |
| C17 | Capítulo 17 — Projeto Final
Integração das técnicas do curso em um projeto completo, com apresentação. |
| <hr/> | |

Referências —

Apresentação

Esta apostila é o material de apoio teórico da disciplina Visão Computacional Aplicada. Ela acompanha as 17 aulas práticas de 120 minutos ministradas em laboratório e reúne, para cada tema, a base conceitual necessária para acompanhar as aulas e revisar o conteúdo depois delas.

A disciplina de Visão Computacional Aplicada visa capacitar os alunos com conhecimentos teóricos e práticos sobre técnicas de visão computacional e suas aplicações em sistemas de software. Ao longo do curso, trabalhamos com algoritmos de processamento de imagem, reconhecimento de padrões e análise visual, desenvolvendo projetos que aplicam essas tecnologias em cenários do mundo real.

Objetivos da disciplina

- Compreender os fundamentos da Visão Computacional e suas principais técnicas.
- Aplicar algoritmos de Visão Computacional em projetos práticos.
- Desenvolver habilidades em processamento de imagem e análise de dados visuais.
- Explorar aplicações de Visão Computacional em Inteligência Artificial, automação e interação homem-máquina.

Abordagem do curso

Diferente da maioria dos cursos de Visão Computacional, que usam C++ com OpenCV nativo, esta disciplina é 100% prática e roda inteiramente no navegador: usamos HTML5, CSS3, JavaScript e OpenCV.js (a versão do OpenCV compilada para WebAssembly), com JavaScript puro — sem frameworks ou bibliotecas de terceiros para manipular a página. Isso elimina a barreira de instalação e compilação, e mantém o foco nos conceitos e algoritmos desde a primeira aula.

Como material de apoio, esta apostila também apresenta, para os principais exemplos, versões equivalentes em Python (com OpenCV-Python) e em Node.js (com OpenCV para Node.js), para que você reconheça os mesmos conceitos em outras linguagens comumente usadas no mercado.

Como usar esta apostila

Cada capítulo desta apostila corresponde a uma aula prática de 120 minutos e segue a mesma estrutura:

1. Teoria — os conceitos e algoritmos do tema, explicados de forma objetiva.
2. Exemplo comentado — um exemplo completo de código, comentado linha a linha, nas três linguagens: HTML5/CSS/JavaScript + OpenCV.js (foco principal), Python + OpenCV-Python, e Node.js + OpenCV para Node.js.
3. Exercícios propostos — atividades para praticar o conteúdo, com dificuldade crescente.
4. Resumo do capítulo — uma síntese rápida para revisão antes de provas ou projetos.

Os blocos de código como o exemplo abaixo indicam trechos prontos para copiar e executar:

```
exemplo.js
console.log("Assim aparece um bloco de código nesta apostila.");
```

Seção 0 — Preparando o Ambiente de Desenvolvimento

Antes da Aula 1, prepare seu ambiente de desenvolvimento. Você vai precisar de três ambientes: o ambiente Web (usado na maioria das aulas), e os ambientes Python e Node.js (usados nos exemplos complementares desta apostila).

0.1 Ferramentas necessárias

- Navegador atualizado: Google Chrome, Microsoft Edge ou Mozilla Firefox.
- Editor de código: Visual Studio Code (recomendado, gratuito) — code.visualstudio.com
- Extensão Live Server para o VS Code (recarrega a página automaticamente).
- Git (opcional, mas recomendado para versionar seus projetos).
- Python 3.10 ou superior — python.org
- Node.js LTS (versão de suporte de longo prazo) — nodejs.org

0.2 Ambiente Web (HTML5 + CSS3 + JavaScript + OpenCV.js)

É o ambiente principal do curso — não exige instalação de bibliotecas: o OpenCV.js é carregado diretamente no navegador via CDN, e todo o restante do código usa apenas JavaScript puro (a API nativa do navegador: `querySelector`, `addEventListener`, `fetch`, etc.), sem frameworks ou bibliotecas de terceiros para manipular a página.

Passo a passo

1. Instale o VS Code e a extensão Live Server.
2. Crie uma pasta para o projeto (ex.: `visao-computacional/`).

3. Dentro dela, crie index.html, a pasta css/ com style.css e a pasta js/ com script.js.
4. No <head> do index.html, importe o OpenCV.js:

```
index.html
<script async src="https://docs.opencv.org/4.x/opencv.js"></script>
```

5. Clique com o botão direito no index.html e escolha "Open with Live Server".
6. Teste se o OpenCV.js carregou, abrindo o Console do navegador (F12):

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  console.log("OpenCV.js pronto! Versão:", cv.getBuildInformation ? "ok" : cv);
};
```

Atenção

O OpenCV.js só fica disponível depois que o WebAssembly termina de carregar. Todo código que usa "cv." deve ficar dentro de cv['onRuntimeInitialized'].

0.3 Ambiente Python

1. Instale o Python 3.10+ (em Windows, marque a opção "Add python.exe to PATH" no instalador).
2. Verifique a instalação:

```
terminal
python --version
```

3. (Recomendado) crie um ambiente virtual para o curso:

```
terminal
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate
```

4. Instale as bibliotecas usadas no curso:

```
terminal
pip install opencv-python numpy matplotlib
```

5. Teste a instalação:

```
terminal
python -c "import cv2; print(cv2. version )"
```

0.4 Ambiente Node.js

1. Instale o Node.js LTS e verifique:

```
terminal
node --version
npm --version
```

2. Crie a pasta do projeto Node e inicialize:

```
terminal
mkdir vc-node && cd vc-node
npm init -y
```

3. Instale o binding de OpenCV para Node.js:

```
terminal
npm install opencv4nodejs
```

Nota sobre o opencv4nodejs

Esse pacote compila o OpenCV nativamente durante a instalação, o que exige Python, CMake e um compilador C++ instalados (no Windows: "Build Tools for Visual Studio"; no Linux: build-essential). Se a instalação falhar, uma alternativa sem compilação nativa é o pacote `@techstark/opencv-js` (a mesma versão WebAssembly usada no navegador, rodando em Node).

4. Teste a instalação:

```
terminal
node -e "const cv = require('opencv4nodejs'); console.log('OpenCV', cv.version);"
```

0.5 Estrutura de pastas recomendada para o curso

```
projeto/
visao-computacional/
├── web/                (HTML5 + CSS + JS + OpenCV.js — usado em quase todas as aulas)
│   ├── aula01/
│   ├── aula02/
│   └── ...
├── python/            (exemplos e exercícios em Python)
│   ├── aula01.py
│   └── ...
└── node/              (exemplos e exercícios em Node.js)
    ├── aula01.js
    └── ...
```

Capítulo 1 — Introdução à Visão Computacional e Inteligência Artificial

Corresponde à Aula 1 (120 minutos).

1.1 Processamento de imagens x Visão Computacional

Embora costumem ser tratadas como sinônimos, processamento de imagens e visão computacional resolvem problemas diferentes. No processamento de imagens, tanto a entrada quanto a saída do processo são imagens: reduzir ruído, ajustar brilho e contraste, aumentar a resolução, rotacionar ou recortar uma imagem são exemplos típicos.

Já na visão computacional, a entrada é uma imagem, mas a saída é uma informação extraída dela: ler o número de uma placa de veículo, verificar se duas fotos pertencem à mesma pessoa, localizar rostos em uma cena ou identificar se um exame de imagem indica uma doença são exemplos de visão computacional.

Existe ainda um terceiro caminho, na direção oposta: a computação gráfica e a IA generativa partem de uma informação (uma descrição, uma cena 3D) e geram uma imagem como saída — por exemplo, quando pedimos a um modelo de IA generativa para criar uma imagem a partir de um texto.

1.2 Inteligência Artificial, Aprendizado de Máquina e Aprendizado Profundo

Esses três termos formam uma hierarquia. Inteligência Artificial (IA) é a área mais ampla: qualquer técnica que faça um computador se comportar de forma "inteligente". Aprendizado de Máquina (Machine Learning) é uma subárea da IA em que o computador aprende um modelo a partir de dados, em vez de seguir regras escritas manualmente. Aprendizado Profundo (Deep Learning) é uma subárea do Aprendizado de Máquina baseada em redes neurais com muitas camadas — as Redes Neurais Convolucionais (CNNs) são o tipo de rede profunda mais usado em visão computacional.

Existem essencialmente duas formas de dar conhecimento a um computador:

- Design direto: um programador escreve as regras manualmente, tipicamente com instruções condicionais (if/else). É a forma clássica de programar.
- Aprendizado de máquina: o computador recebe exemplos de entrada e saída e aprende sozinho o modelo que relaciona um ao outro.

Um bom exemplo da diferença entre as duas abordagens é tentar escrever um programa que classifique uma foto de rosto em "masculino" ou "feminino" usando apenas regras se/então. É praticamente impossível descrever essa regra em código — mas, com aprendizado de máquina, basta treinar um modelo com centenas de fotos de cada categoria para obter um classificador com erro de aproximadamente 1%.

Até por volta de 2010, o aprendizado de máquina clássico resolvia apenas a etapa de classificação: um ser humano ainda precisava escrever manualmente as funções que extraíam os atributos relevantes da imagem (por exemplo, para diagnosticar uma lesão de pele: assimetria, irregularidade da borda, variação de cor). A grande virada do aprendizado profundo foi automatizar também a extração de atributos — o próprio algoritmo aprende, a partir dos exemplos, quais características observar na imagem.

1.3 Um breve histórico

- 1950 — Alan Turing propõe o Teste de Turing, um critério para avaliar se uma máquina "pensa".
- Década de 1980 — início da popularização do Aprendizado de Máquina.
- Por volta de 2010 — surgem as primeiras redes neurais profundas aplicadas com sucesso à visão computacional.
- A partir de 2015 — o aprendizado profundo se populariza em larga escala, resolvendo problemas antes considerados insolúveis.
- 2022 em diante — explosão da IA generativa (ChatGPT, Gemini, Claude e outros), que gera texto, imagem, áudio e código a partir de descrições.

1.4 Aplicações da Visão Computacional

Ao longo deste curso veremos, na prática, diversas aplicações. Entre as mais relevantes:

- Classificação de imagens — decidir a que categoria uma imagem pertence (ex.: diagnosticar uma radiografia, identificar uma espécie, classificar grãos).
- Detecção de objetos — localizar onde cada objeto está na imagem, indicando posição e escala.
- Segmentação semântica — classificar cada pixel da imagem, separando objetos de interesse do fundo.
- Reconhecimento facial — comparar um rosto capturado com um banco de referências para identificar a pessoa.
- Colorização automática — converter fotos em preto-e-branco para coloridas, aprendendo com milhares de exemplos.
- Super-resolução — aumentar a resolução de uma imagem, reconstruindo detalhes plausíveis.

1.5 Por que programar Visão Computacional no navegador?

Nesta disciplina, a maior parte dos exemplos e exercícios roda inteiramente no navegador, usando HTML5, CSS3, JavaScript puro e OpenCV.js. As vantagens dessa abordagem:

- Zero instalação pesada — sem compilar C++ nem configurar um ambiente complexo, basta um navegador atualizado.
- Multiplataforma — o mesmo código roda em Windows, macOS, Linux, tablets e celulares.

- Fácil de compartilhar — um projeto em HTML/JS pode ser publicado com um link, ideal para portfólio.
- Performance real — o OpenCV.js usa WebAssembly, tecnologia que executa código compilado (originalmente em C++) dentro do navegador com desempenho próximo ao nativo.

1.6 Conhecendo o OpenCV.js

O OpenCV é a biblioteca de visão computacional mais usada do mundo, escrita originalmente em C++. O OpenCV.js é sua versão compilada para WebAssembly, o que permite executar, dentro do navegador, praticamente os mesmos algoritmos usados em produção por empresas ao redor do mundo.

Alguns elementos fundamentais da API do OpenCV.js:

- `cv.Mat` — a estrutura de dados que representa uma imagem como uma matriz de pixels.
- `cv.imread(elemento)` — lê uma imagem a partir de um elemento `<img>` ou `<canvas>` e retorna um `cv.Mat`.
- `cv.imshow(idDoCanvas, mat)` — desenha um `cv.Mat` em um elemento `<canvas>`.
- `cv['onRuntimeInitialized']` — callback disparado quando o WebAssembly termina de carregar; todo código que usa a API `cv` deve ficar dentro dele.

Boas práticas

Todo `cv.Mat` criado manualmente (com `new cv.Mat()`) precisa ser liberado chamando `.delete()` quando não for mais necessário — o OpenCV.js gerencia sua própria memória fora do garbage collector do JavaScript.

1.7 Exemplo comentado: Hello Vision

Este primeiro exemplo lê uma imagem escolhida pelo usuário, exibe a versão original e, ao clicar em um botão, mostra a conversão para escala de cinza.

1.7.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
index.html
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8" />
  <title>Aula 1 · Hello Vision</title>
  <script async src="https://docs.opencv.org/4.x/opencv.js"></script>
</head>
<body>
  <h1>Aula 1 · Hello Vision</h1>
  <input type="file" id="inputImagem" accept="image/*" />
  <button id="btnCinza">Converter p/ cinza</button>
  <canvas id="canvasOriginal"></canvas>
  <canvas id="canvasSaida"></canvas>
  <script src="js/script.js"></script>
</body>
```

```

</html>
js/script.js
// só usamos o OpenCV depois do WebAssembly carregar
cv['onRuntimeInitialized'] = function () {
  const inputImagem = document.querySelector('#inputImagem');
  const btnCinza = document.querySelector('#btnCinza');
  let src;

  inputImagem.addEventListener('change', function (e) {
    const img = document.createElement('img');
    img.src = URL.createObjectURL(e.target.files[0]);
    img.onload = () => {
      src = cv.imread(img);
      cv.imshow("canvasOriginal", src);
    };
  });

  btnCinza.addEventListener('click', function () {
    let cinza = new cv.Mat();
    cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);
    cv.imshow("canvasSaida", cinza);
    cinza.delete();
  });
};

```

1.7.2 Versão Python — OpenCV-Python

O mesmo conceito, em um script Python executado localmente:

```

aula01.py
import cv2

# Carrega a imagem colorida (BGR, convenção do OpenCV)
img = cv2.imread("flor.jpg")

# Converte para escala de cinza
cinza = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Exibe as duas imagens em janelas
cv2.imshow("Original", img)
cv2.imshow("Escala de cinza", cinza)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Salva o resultado em disco
cv2.imwrite("flor_cinza.jpg", cinza)

```

1.7.3 Versão Node.js — opencv4nodejs

E a versão equivalente rodando em Node.js, fora do navegador:

```

aula01.js
const cv = require('opencv4nodejs');

// Carrega a imagem colorida (BGR)
const img = cv.imread('flor.jpg');

```

```
// Converte para escala de cinza
const cinza = img.cvtColor(cv.COLOR_BGR2GRAY);

// Salva o resultado em disco
cv.imwrite('flor_cinza.jpg', cinza);

// (Opcional) exibe em uma janela
cv.imshow('Original', img);
cv.imshow('Escala de cinza', cinza);
cv.waitKey();
```

Comparando as três versões

Repare que a operação central — converter para escala de cinza — é conceitualmente idêntica nas três linguagens: `cv.cvtColor(origem, destino, cv.COLOR_..2GRAY)` no navegador, `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` em Python e `img.cvtColor(cv.COLOR_BGR2GRAY)` em Node.js. O que muda é a forma de obter e mostrar a imagem em cada ambiente.

1.8 Erros comuns

- "cv is not defined" — o código tentou usar o OpenCV.js antes do WebAssembly terminar de carregar; sempre escreva dentro de `cv['onRuntimeInitialized']`.
- Canvas em branco — o id do `<canvas>` no HTML deve ser idêntico ao usado em `cv.imshow("id", ...)`.
- Vazamento de memória no navegador — todo `cv.Mat()` criado manualmente precisa ser liberado com `.delete()`.
- Abrir o `index.html` direto pelo sistema de arquivos (`file:///`) pode bloquear a leitura de imagens; use sempre um servidor local, como o Live Server.

1.9 Exercícios propostos

1. Reproduza o exemplo Hello Vision na versão Web, usando uma imagem à sua escolha.
2. Adicione um segundo botão que aplique `cv.Canny()` para detectar bordas na imagem.
3. Implemente a mesma conversão para escala de cinza em Python, usando uma imagem diferente da usada em aula.
4. (Desafio) Usando `getUserMedia()` para acessar a webcam, capture frames em um `<canvas>` e aplique, com OpenCV.js, uma conversão de cor em tempo real.

1.10 Resumo do capítulo

- Processamento de imagens transforma imagem em imagem; visão computacional transforma imagem em informação.
- IA $\supset$ Aprendizado de Máquina $\supset$ Aprendizado Profundo (CNNs).
- Design direto usa regras escritas manualmente; aprendizado de máquina aprende o modelo a partir de exemplos.

- O curso é 100% prático, rodando no navegador com HTML5 + CSS3 + JavaScript puro + OpenCV.js (WebAssembly), com paralelos em Python e Node.js.
- `cv.Mat`, `cv.imread`, `cv.imshow` e `cv['onRuntimeInitialized']` são a base da API do OpenCV.js.

Capítulo 2 — Fundamentos de Processamento de Imagens

Corresponde à Aula 2 (120 minutos).

2.1 O que é um pixel

Uma imagem digital é uma matriz de números. Cada elemento dessa matriz é um pixel ("picture element"), a menor unidade de informação visual da imagem. Em uma imagem em escala de cinza, cada pixel guarda um único número, normalmente entre 0 (preto) e 255 (branco), representado em 8 bits. Em uma imagem colorida, cada pixel guarda três (ou quatro) números — um para cada canal de cor.

O tamanho de uma imagem é descrito pelo número de linhas (altura) e colunas (largura) dessa matriz. Uma imagem de 1920×1080 pixels tem, portanto, 1920×1080 = 2.073.600 pixels — e, se for colorida (3 canais), mais de 6 milhões de números armazenados.

2.2 Canais de cor: RGB e BGR

O modelo de cor mais comum é o RGB (Red, Green, Blue — vermelho, verde e azul). Cada pixel colorido é descrito pela intensidade desses três canais, e a combinação deles forma qualquer cor visível. Por exemplo, (255, 0, 0) é vermelho puro, (0, 255, 0) é verde puro e (255, 255, 255) é branco.

Atenção: RGB x BGR

O OpenCV, por razões históricas, lê e representa imagens coloridas na ordem BGR (azul, verde, vermelho) e não RGB. Isso costuma confundir iniciantes: se uma imagem aparecer com as cores "troçadas" (azul e vermelho invertidos), normalmente é porque uma conversão de canal foi esquecida. No navegador, o `cv.imread()` a partir de um <canvas> já devolve os dados no formato RGBA (por causa do Canvas API), então a conversão mais comum em OpenCV.js é `cv.COLOR_RGBA2GRAY` ou `cv.COLOR_RGBA2BGR`, dependendo do que se precisa a seguir.

2.3 Escala de cinza

Converter uma imagem colorida para escala de cinza reduz os três canais a um único valor de luminância por pixel. A conversão não é uma simples média aritmética dos três canais — o olho humano é mais sensível ao verde do que ao vermelho e ao azul, então a fórmula usada pelo OpenCV pondera os canais de forma desigual:

```
fórmula
cinza = 0.299 × R + 0.587 × G + 0.114 × B
```

Essa fórmula reflete a sensibilidade do olho humano: um verde puro parece muito mais "claro" do que um azul puro de mesma intensidade numérica, e a conversão para cinza precisa preservar essa percepção de luminosidade.

2.4 RGB, CMY e HSV: diferentes formas de descrever a mesma cor

RGB é um modelo aditivo (útil para telas, que emitem luz): somar todos os canais no máximo gera branco. Existe também o modelo subtrativo CMY (Ciano, Magenta, Amarelo), usado em impressão: nele, misturar todas as tintas no máximo tende ao preto, porque as tintas absorvem luz em vez de emití-la.

Para tarefas de visão computacional, um modelo particularmente útil é o HSV (Hue, Saturation, Value — matiz, saturação e valor):

- Matiz (Hue): a "cor" propriamente dita (vermelho, verde, azul...), representada como um ângulo de 0° a 360° numa roda de cores.
- Saturação (Saturation): o quão "pura" ou "viva" é a cor — baixa saturação tende ao cinza, alta saturação é uma cor vibrante.
- Valor (Value): o brilho da cor — de escuro (0) a claro (máximo).

A grande vantagem do HSV para visão computacional é separar a informação de COR (matiz) da informação de ILUMINAÇÃO (valor). Isso torna a segmentação por cor muito mais robusta a variações de luz do que trabalhar diretamente em RGB — um objeto vermelho continua tendo matiz "vermelho" tanto sob luz forte quanto sob sombra, mesmo que seu valor (brilho) mude bastante.

2.5 A estrutura de uma imagem no OpenCV (cv.Mat)

No OpenCV (e no OpenCV.js), uma imagem é representada pela estrutura Mat — uma matriz N-dimensional que guarda, além dos valores de pixel, metadados como número de linhas, colunas e canais, e o tipo de dado de cada pixel (ex.: 8 bits sem sinal, ponto flutuante de 32 bits).

```
js/script.js
let src = cv.imread('minhaImagem');
console.log('linhas (altura):', src.rows);
console.log('colunas (largura):', src.cols);
console.log('canais:', src.channels()); // 4 = RGBA, 1 = escala de cinza
```

2.6 Exemplo comentado: espaços de cor

2.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  const img = document.querySelector('#minhaImagem');
  let src = cv.imread(img);

  // RGBA -> escala de cinza
  let cinza = new cv.Mat();
  cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);
  cv.imshow('canvasCinza', cinza);

  // RGBA -> HSV (primeiro remove o canal alfa)
  let rgb = new cv.Mat(), hsv = new cv.Mat();
  cv.cvtColor(src, rgb, cv.COLOR_RGBA2RGB);
```

```

cv.cvtColor(rgb, hsv, cv.COLOR_RGB2HSV);
cv.imshow('canvasHSV', hsv);

// separar canais
let canais = new cv.MatVector();
cv.split(rgb, canais);
cv.imshow('canvasCanalR', canais.get(0));
cv.imshow('canvasCanalG', canais.get(1));
cv.imshow('canvasCanalB', canais.get(2));

cinza.delete(); rgb.delete(); hsv.delete(); canais.delete();
};

```

2.6.2 Versão Python — OpenCV-Python

```

aula02.py
import cv2

img = cv2.imread("flor.jpg") # lido em BGR

cinza = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# separar canais B, G, R
b, g, r = cv2.split(img)

cv2.imwrite("flor_cinza.jpg", cinza)
cv2.imwrite("flor_hsv.jpg", hsv)
cv2.imwrite("flor_r.jpg", r)

```

2.6.3 Versão Node.js — opencv4nodejs

```

aula02.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg'); // BGR

const cinza = img.cvtColor(cv.COLOR_BGR2GRAY);
const hsv = img.cvtColor(cv.COLOR_BGR2HSV);

const [b, g, r] = img.splitChannels();

cv.imwrite('flor_cinza.jpg', cinza);
cv.imwrite('flor_hsv.jpg', hsv);
cv.imwrite('flor_r.jpg', r);

```

2.7 Erros comuns

- Esquecer que `cv.imread()` no navegador retorna RGBA (4 canais) e tentar converter direto para HSV — é necessário primeiro remover o canal alfa com `cv.COLOR_RGBA2RGB`.
- Confundir a ordem BGR (OpenCV nativo/Python/Node) com RGB (a maioria das outras ferramentas), gerando imagens com cores trocadas.

- Achar que HSV é sempre representado com H de 0 a 360 — no OpenCV, para caber em 8 bits, H vai de 0 a 179 (metade do valor em graus).

2.8 Exercícios propostos

1. Carregue uma imagem colorida e exiba, lado a lado, seus três canais RGB isolados.
2. Converta a mesma imagem para HSV e exiba separadamente os canais H, S e V.
3. Escreva uma função que recebe uma imagem e devolve o negativo dela ($255 - \text{valor}$) em cada canal.
4. (Desafio) Compare visualmente a robustez de segmentar uma cor específica usando limiares em RGB versus limiares em HSV, sob duas fotos da mesma cena com iluminações diferentes.

2.9 Resumo do capítulo

- Um pixel é a menor unidade de uma imagem digital; imagens coloridas guardam vários canais por pixel.
- RGB é aditivo (telas); CMY é subtrativo (impressão); HSV separa cor (matiz) de iluminação (valor).
- A conversão para escala de cinza pondera os canais de forma desigual, refletindo a sensibilidade do olho humano.
- O OpenCV usa a ordem BGR por padrão — no navegador, imagens são lidas como RGBA.
- HSV é preferível a RGB para segmentar objetos por cor, por ser mais robusto a variações de iluminação.

Capítulo 3 — Operações Básicas de Imagem

Corresponde à Aula 3 (120 minutos).

3.1 Imagens como matrizes de números

Como vimos no Capítulo 2, uma imagem é uma matriz de números. Isso significa que podemos aplicar sobre ela as mesmas operações que aplicaríamos sobre qualquer matriz: soma, subtração, operações lógicas bit a bit, e transformações ponto a ponto (aplicadas a cada pixel individualmente, sem olhar para os vizinhos).

3.2 Operações aritméticas entre imagens

`cv.add()` e `cv.subtract()` somam ou subtraem duas imagens de mesmo tamanho, pixel a pixel. Um detalhe importante é a saturação: se a soma de dois pixels ultrapassar 255 (o valor máximo em 8 bits), o resultado é "grampeado" em 255, em vez de "dar a volta" (overflow). Da mesma forma, uma subtração que resultaria em um valor negativo é grampeada em 0.

```
conceito
// exemplo numérico: soma com saturação
200 + 100 = 300 -> grampeado para 255 (não 44, como seria num overflow real)
50 - 100 = -50 -> grampeado para 0
```

Também é possível somar duas imagens com pesos diferentes (uma média ponderada), usando `cv.addWeighted()` — muito útil para criar efeitos de transição (fade) ou combinar duas imagens (ex.: uma imagem original com uma versão borrada, para um efeito artístico).

```
fórmula do addWeighted
resultado =  $\alpha \cdot \text{imagem1} + \beta \cdot \text{imagem2} + \gamma$ 
```

3.3 Operações lógicas (bit a bit)

As operações lógicas AND, OR, XOR e NOT são aplicadas bit a bit sobre os valores dos pixels. Em visão computacional, seu uso mais comum é operar sobre máscaras binárias (imagens onde cada pixel é 0 ou 255):

- AND (`cv.bitwise_and`): mantém apenas os pixels onde AMBAS as imagens são "brancas" — usado para "recortar" uma região definida por uma máscara.
- OR (`cv.bitwise_or`): mantém os pixels onde QUALQUER uma das imagens é "branca" — usado para combinar duas máscaras.
- XOR (`cv.bitwise_xor`): mantém os pixels onde exatamente UMA das imagens é "branca" — útil para encontrar diferenças entre duas máscaras.
- NOT (`cv.bitwise_not`): inverte os pixels — o que era 0 vira 255, e vice-versa (é assim que se calcula o negativo de uma imagem binária).

3.4 Brilho e contraste

Brilho e contraste podem ser ajustados com uma transformação linear simples, aplicada a cada pixel:

```
fórmula
novo pixel =  $\alpha \cdot \text{pixel original} + \beta$ 
```

Nessa fórmula, β (beta) controla o BRILHO — somar um valor positivo clareia a imagem inteira, e um valor negativo a escurece. Já α (alfa) controla o CONTRASTE — valores de α maiores que 1 aumentam a diferença entre tons claros e escuros; valores entre 0 e 1 reduzem essa diferença, achatando a imagem.

O OpenCV oferece a função `cv.convertScaleAbs()`, que aplica exatamente essa fórmula, já cuidando da saturação em 0 e 255.

3.5 Região de interesse (ROI)

Frequentemente, só precisamos processar uma parte da imagem, não ela inteira — isso é chamado de região de interesse (Region of Interest, ROI). Um `cv.Rect` define um retângulo (posição x, y, largura e altura), e `cv.Mat.roi()` extrai essa região sem copiar os dados (é uma "visão" sobre a mesma memória, o que a torna muito rápida).

Atenção

Como o ROI compartilha memória com a imagem original, modificar os pixels do ROI também modifica a imagem original naquela região. Se você precisa de uma cópia independente, use `.clone()` sobre o ROI.

3.6 Exemplo comentado: aritmética, lógica e brilho/contraste

3.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');

  // brilho (+50) e contraste (x1.3)
  let ajustada = new cv.Mat();
  cv.convertScaleAbs(src, ajustada, 1.3, 50);
  cv.imshow('canvasAjustada', ajustada);

  // região de interesse: recorta um retângulo
  let rect = new cv.Rect(50, 50, 150, 150);
  let roi = src.roi(rect);
  cv.imshow('canvasROI', roi);

  // máscara: mantém só a região dentro de um círculo
  let mascara = cv.Mat.zeros(src.rows, src.cols, cv.CV_8UC1);
  let centro = new cv.Point(src.cols / 2, src.rows / 2);
  cv.circle(mascara, centro, 100, new cv.Scalar(255), -1);
  let recorte = new cv.Mat();
  cv.bitwise_and(src, src, recorte, mascara);
```

```

cv.imshow('canvasRecorte', recorte);

ajustada.delete(); mascara.delete(); recorte.delete();
};

```

3.6.2 Versão Python — OpenCV-Python

```

aula03.py
import cv2
import numpy as np

img = cv2.imread("flor.jpg")

# brilho (+50) e contraste (x1.3)
ajustada = cv2.convertScaleAbs(img, alpha=1.3, beta=50)

# região de interesse
roi = img[50:200, 50:200] # linhas 50-200, colunas 50-200

# máscara circular
mascara = np.zeros(img.shape[:2], dtype=np.uint8)
centro = (img.shape[1] // 2, img.shape[0] // 2)
cv2.circle(mascara, centro, 100, 255, -1)
recorte = cv2.bitwise_and(img, img, mask=mascara)

cv2.imwrite("flor_ajustada.jpg", ajustada)
cv2.imwrite("flor_recorte.jpg", recorte)

```

3.6.3 Versão Node.js — opencv4nodejs

```

aula03.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg');

// brilho e contraste: multiplica e soma diretamente na matriz
const ajustada = img.convertTo(cv.CV_8U, 1.3, 50);

// região de interesse
const rect = new cv.Rect(50, 50, 150, 150);
const roi = img.getRegion(rect);

// máscara circular
let mascara = new cv.Mat(img.rows, img.cols, cv.CV_8UC1, 0);
const centro = new cv.Point2(img.cols / 2, img.rows / 2);
mascara.drawCircle(centro, 100, new cv.Vec3(255, 255, 255), -1);
const recorte = img.copy(mascara);

cv.imwrite('flor_ajustada.jpg', ajustada);
cv.imwrite('flor_recorte.jpg', recorte);

```

3.7 Erros comuns

- Somar/subtrair duas imagens de tamanhos diferentes — todas as operações aritméticas e lógicas exigem que as imagens tenham exatamente as mesmas dimensões e o mesmo tipo de dado.
- Esquecer que `cv.add()` satura em 255 (diferente de uma soma "crua" em JavaScript, que poderia dar um valor fora da faixa esperada).
- Modificar um ROI achando que está trabalhando numa cópia — lembre-se de que ele compartilha memória com a imagem original.

3.8 Exercícios propostos

1. Implemente um controle deslizante (slider) HTML que ajusta o brilho de uma imagem em tempo real, usando `cv.convertScaleAbs()`.
2. Recorte uma região retangular de uma foto e salve-a como uma imagem separada.
3. Combine duas imagens de mesmo tamanho com `cv.addWeighted()`, variando α de 0 a 1, criando um efeito de transição (fade).
4. (Desafio) Use uma máscara em forma de texto (desenhado com `cv.putText()`) para "recortar" uma foto no formato de uma palavra.

3.9 Resumo do capítulo

- Operações aritméticas entre imagens (`cv.add`, `cv.subtract`) saturam em 0 e 255, em vez de estourar (overflow).
- `cv.addWeighted()` combina duas imagens com pesos, útil para transições e efeitos artísticos.
- Operações lógicas (AND, OR, XOR, NOT) trabalham bit a bit e são a base para aplicar máscaras.
- Brilho e contraste seguem a fórmula $\text{novo_pixel} = \alpha \cdot \text{pixel} + \beta$, aplicada com `cv.convertScaleAbs()`.
- ROI (região de interesse) permite processar só uma parte da imagem, mas compartilha memória com a imagem original.

Capítulo 4 — Histogramas, Limiarização e Binarização

Corresponde à Aula 4 (120 minutos).

4.1 O que é um histograma de imagem

O histograma de uma imagem em escala de cinza é um gráfico de barras que mostra quantos pixels existem para cada nível de intensidade, de 0 a 255. Ele resume, de forma compacta, a distribuição de tons da imagem: uma imagem escura concentra o histograma à esquerda (valores baixos); uma imagem clara concentra à direita; uma imagem de baixo contraste tem um histograma "estreito", concentrado numa faixa pequena de valores; uma imagem de alto contraste tem um histograma "espalhado" por toda a faixa.

O histograma é fundamental para decidir automaticamente parâmetros de processamento de imagem, como o limiar de binarização que veremos a seguir.

4.2 Equalização de histograma

A equalização de histograma é uma técnica que redistribui os níveis de cinza de uma imagem para que o histograma resultante fique o mais "achatado" (uniforme) possível, o que geralmente aumenta o contraste global da imagem — muito útil para melhorar fotos subexpostas ou superexpostas.

```
conceito
cv.equalizeHist(origem, destino); // só funciona em imagens de 1 canal (escala de cinza)
```

4.3 Limiarização (thresholding)

Limiarizar (ou binarizar) uma imagem é transformá-la em uma imagem binária (apenas dois valores: 0 e 255), com base em um limiar T : pixels acima de T viram uma cor, pixels abaixo viram outra. É uma das operações mais usadas em visão computacional, pois simplifica drasticamente uma imagem antes de segmentá-la ou analisá-la.

O OpenCV oferece vários TIPOS de limiarização, todos aplicados através da mesma função `cv.threshold()`:

- `THRESH_BINARY`: $\text{pixel} > T$ vira o valor máximo (geralmente 255); caso contrário, vira 0.
- `THRESH_BINARY_INV`: o inverso do anterior — $\text{pixel} > T$ vira 0; caso contrário, vira o valor máximo.
- `THRESH_TRUNC`: $\text{pixel} > T$ é "truncado" para o valor de T ; pixels $\leq T$ permanecem inalterados.
- `THRESH_TOZERO`: $\text{pixel} > T$ permanece inalterado; pixels $\leq T$ viram 0.

4.4 Como escolher o limiar automaticamente: o método de Otsu

Escolher manualmente o valor de T pode ser trabalhoso e pouco robusto — o limiar ideal muda de imagem para imagem, dependendo da iluminação. O método de Otsu resolve esse problema automaticamente: ele analisa o histograma da imagem e encontra o limiar que melhor separa dois grupos de pixels (dois "picos" do histograma), minimizando a variância dentro de cada grupo e maximizando a variância entre os grupos.

```
conceito
cv.threshold(origem, destino, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU);
// o valor 0 é ignorado — o Otsu calcula o T ideal automaticamente
```

Quando o Otsu funciona bem

O método de Otsu funciona melhor quando o histograma da imagem é bimodal, isto é, tem dois "picos" bem separados (por exemplo, um objeto claro sobre um fundo escuro). Em imagens com iluminação muito irregular, um único limiar global pode não ser suficiente.

4.5 Limiarização adaptativa

Quando a iluminação da imagem varia muito de uma região para outra (por exemplo, uma foto de um documento com uma sombra de um lado), um único limiar global pode falhar: uma parte da imagem fica toda branca e outra toda preta. A limiarização adaptativa resolve isso calculando um limiar diferente para CADA região da imagem, com base na vizinhança local de cada pixel (geralmente sua média ou média ponderada gaussiana).

```
conceito
cv.adaptiveThreshold(origem, destino, 255,
    cv.ADAPTIVE_THRESH_GAUSSIAN_C, cv.THRESH_BINARY,
    blockSize, // tamanho da vizinhança considerada (deve ser ímpar)
    C); // constante subtraída da média da vizinhança
```

4.6 Exemplo comentado: histograma e limiarização

4.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
    let src = cv.imread('minhaImagem');
    let cinza = new cv.Mat();
    cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);

    // equalização de histograma
    let equalizada = new cv.Mat();
    cv.equalizeHist(cinza, equalizada);
    cv.imshow('canvasEqualizada', equalizada);

    // limiarização automática (Otsu)
    let binaria = new cv.Mat();
    cv.threshold(cinza, binaria, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU);
```

```

cv.imshow('canvasBinaria', binaria);

// limiarização adaptativa (para iluminação irregular)
let adaptativa = new cv.Mat();
cv.adaptiveThreshold(cinza, adaptativa, 255,
    cv.ADAPTIVE_THRESH_GAUSSIAN_C, cv.THRESH_BINARY, 15, 5);
cv.imshow('canvasAdaptativa', adaptativa);

cinza.delete(); equalizada.delete(); binaria.delete(); adaptativa.delete();
};

```

4.6.2 Versão Python — OpenCV-Python

```

aula04.py
import cv2

img = cv2.imread("documento.jpg", cv2.IMREAD_GRAYSCALE)

equalizada = cv2.equalizeHist(img)

_, binaria = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

adaptativa = cv2.adaptiveThreshold(img, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 15, 5)

cv2.imwrite("doc_equalizado.jpg", equalizada)
cv2.imwrite("doc_binario.jpg", binaria)
cv2.imwrite("doc_adaptativo.jpg", adaptativa)

```

4.6.3 Versão Node.js — opencv4nodejs

```

aula04.js
const cv = require('opencv4nodejs');

const img = cv.imread('documento.jpg', cv.IMREAD_GRAYSCALE);

const equalizada = img.equalizeHist();

const { dst: binaria } = cv.threshold(img, 0, 255,
    cv.THRESH_BINARY + cv.THRESH_OTSU);

const adaptativa = img.adaptiveThreshold(255,
    cv.ADAPTIVE_THRESH_GAUSSIAN_C, cv.THRESH_BINARY, 15, 5);

cv.imwrite('doc_equalizado.jpg', equalizada);
cv.imwrite('doc_binario.jpg', binaria);
cv.imwrite('doc_adaptativo.jpg', adaptativa);

```

4.7 Erros comuns

- Chamar `cv.equalizeHist()` numa imagem colorida — só funciona em imagens de 1 canal; para colorir, aplique em um canal de luminância (ex.: o canal V do HSV) e recombine.

- Usar Otsu em imagens com iluminação muito irregular e esperar um bom resultado — nesse caso, prefira a limiarização adaptativa.
- Esquecer que `blockSize` da limiarização adaptativa precisa ser um número ímpar.

4.8 Exercícios propostos

1. Plote o histograma de uma imagem em escala de cinza (em Python, use `matplotlib`; no navegador, desenhe as barras num `<canvas>`).
2. Compare visualmente o resultado de `THRESH_BINARY`, `THRESH_TRUNC` e `THRESH_TOZERO` na mesma imagem e mesmo limiar.
3. Aplique limiarização com Otsu numa foto tirada em ambiente com iluminação uniforme e depois numa foto com sombra — compare com a limiarização adaptativa.
4. (Desafio) Implemente sua própria versão simplificada do método de Otsu, testando cada limiar possível de 0 a 255 e escolhendo o que minimiza a variância intra-classe.

4.9 Resumo do capítulo

- O histograma mostra a distribuição de intensidades de uma imagem e orienta decisões de processamento.
- A equalização de histograma aumenta o contraste redistribuindo os níveis de cinza.
- `cv.threshold()` com os tipos `BINARY`, `BINARY_INV`, `TRUNC` e `TOZERO` binariza (ou simplifica) uma imagem.
- O método de Otsu escolhe automaticamente o melhor limiar global, ideal para histogramas bimodais.
- A limiarização adaptativa calcula um limiar por região, essencial quando a iluminação da imagem é irregular.

Capítulo 5 — Segmentação: Crescimento de Região e Componentes Conexos

Corresponde à Aula 5 (120 minutos).

5.1 O que é segmentação

Segmentar uma imagem é dividi-la em regiões que compartilham alguma propriedade em comum — tipicamente, separar "objetos" do "fundo". Depois de limiarizar uma imagem (Capítulo 4), obtemos uma imagem binária com pixels "objeto" e pixels "fundo", mas ainda não sabemos quantos objetos distintos existem, nem onde cada um está. É aí que entram os algoritmos de componentes conexos.

5.2 4-conectividade e 8-conectividade

Dois pixels são considerados vizinhos de formas diferentes, dependendo da convenção de conectividade escolhida:

- 4-conectividade: considera vizinhos apenas os pixels acima, abaixo, à esquerda e à direita (conexões "em cruz").
- 8-conectividade: além dos quatro anteriores, também considera os quatro vizinhos na diagonal.

A escolha entre 4 e 8-conectividade pode mudar o resultado da segmentação: dois pixels diagonalmente adjacentes são considerados "do mesmo objeto" na 8-conectividade, mas "de objetos diferentes" na 4-conectividade.

5.3 Por que um algoritmo ingênuo não funciona

Uma primeira ideia para rotular componentes seria percorrer a imagem pixel a pixel (por exemplo, da esquerda para a direita, de cima para baixo) e, ao encontrar um pixel "objeto", copiar o rótulo de algum vizinho já visitado. O problema é que essa abordagem "ingênua" falha em formas complexas — como um "U" ou uma espiral — porque, ao percorrer a imagem numa única passada, pixels de um MESMO objeto podem acabar recebendo rótulos diferentes por ainda não terem sido conectados na hora em que foram visitados.

5.4 Crescimento de região (seed growth)

Uma solução mais robusta é o crescimento de região, também chamado de crescimento de semente. A ideia: escolhemos um pixel "objeto" ainda não rotulado (a semente), damos a ele um novo rótulo, e então "crescemos" esse rótulo para todos os vizinhos que também são "objeto" — que por sua vez crescem para os SEUS vizinhos, e assim sucessivamente, até que não haja mais vizinhos não-rotulados para expandir.

Esse crescimento pode ser implementado de três formas equivalentes em resultado, mas diferentes em ordem de visita:

- Fila (BFS — busca em largura): visita os vizinhos mais próximos da semente primeiro, expandindo "em camadas".
- Pilha (DFS — busca em profundidade, iterativa): aprofunda-se ao máximo por um caminho antes de voltar.
- Recursão (DFS recursiva): a versão mais simples de escrever, mas pode estourar a pilha de chamadas em componentes muito grandes.

Preenchimento por inundação (flood fill)

`cv.floodFill()` é a implementação pronta do OpenCV para crescimento de região a partir de uma semente — muito usada, por exemplo, na ferramenta "balde de tinta" de editores de imagem.

5.5 Rotulação de todos os componentes de uma vez

Quando queremos rotular TODOS os componentes conexos de uma imagem (não apenas crescer uma única semente), usamos `cv.connectedComponents()` (ou `cv.connectedComponentsWithStats()`, que também devolve estatísticas como área, bounding box e centroide de cada componente). Internamente, o algoritmo aplica a mesma ideia de crescimento de região, mas repetida automaticamente para cada pixel ainda não rotulado da imagem.

5.6 Exemplo comentado: componentes conexos

5.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');
  let cinza = new cv.Mat(), binaria = new cv.Mat();
  cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);
  cv.threshold(cinza, binaria, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU);

  // rotula todos os componentes conexos
  let rotulos = new cv.Mat();
  let stats = new cv.Mat(), centroids = new cv.Mat();
  let numComponentes = cv.connectedComponentsWithStats(
    binaria, rotulos, stats, centroids, 8, cv.CV_32S);

  console.log('Componentes encontrados (incluindo o fundo):', numComponentes);
  for (let i = 1; i < numComponentes; i++) { // i=0 é o fundo
    let area = stats.intPtr(i, cv.CC_STAT_AREA)[0];
    console.log('Componente ' + i + ' tem área ' + area + ' pixels');
  }

  // preenchimento por inundação a partir de um ponto (x=50, y=50)
  let mascara = new cv.Mat.zeros(src.rows + 2, src.cols + 2, cv.CV_8UC1);
  let cor = new cv.Scalar(255, 0, 0, 255);
```

```

cv.floodFill(src, mascara, new cv.Point(50, 50), cor);
cv.imshow('canvasFloodFill', src);

cinza.delete(); binaria.delete(); rotulos.delete();
stats.delete(); centroids.delete(); mascara.delete();
};

```

5.6.2 Versão Python — OpenCV-Python

```

aula05.py
import cv2

img = cv2.imread("letras.bmp", cv2.IMREAD_GRAYSCALE)
_, binaria = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

numComponentes, rotulos, stats, centroids = cv2.connectedComponentsWithStats(
    binaria, connectivity=8)

print('Componentes encontrados (incluindo o fundo):', numComponentes)
for i in range(1, numComponentes): # i=0 é o fundo
    area = stats[i, cv2.CC_STAT_AREA]
    print(f'Componente {i} tem área {area} pixels')

# preenchimento por inundação
colorida = cv2.imread('letras.bmp')
mascara = None # None deixa o floodFill criar a máscara internamente em algumas
versões
h, w = colorida.shape[:2]
mascara = cv2.copyMakeBorder(
    (colorida[:, :, 0] * 0).astype('uint8'), 1, 1, 1, 1, cv2.BORDER_CONSTANT, 0)
cv2.floodFill(colorida, mascara, (50, 50), (255, 0, 0))
cv2.imwrite("letras_flood.png", colorida)

```

5.6.3 Versão Node.js — opencv4nodejs

```

aula05.js
const cv = require('opencv4nodejs');

const img = cv.imread('letras.bmp', cv.IMREAD_GRAYSCALE);
const { dst: binaria } = cv.threshold(img, 0, 255,
    cv.THRESH_BINARY + cv.THRESH_OTSU);

// opencv4nodejs expõe contornos como alternativa direta a connectedComponents
const contornos = binaria.findContours(cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE);
console.log('Componentes encontrados:', contornos.length);
contornos.forEach((c, i) => {
    console.log(`Componente ${i} tem área ${c.area} pixels`);
});

```

5.7 Erros comuns

- Esquecer de binarizar a imagem antes de rotular componentes conexos — o algoritmo espera uma imagem de dois valores (0 e 255).

- Confundir o rótulo 0 (fundo) com um componente de verdade ao percorrer os resultados de `connectedComponents`.
- Usar 4-conectividade quando o esperado é 8 (ou vice-versa) e obter uma contagem de objetos diferente da esperada.

5.8 Exercícios propostos

1. Implemente o crescimento de região usando uma fila (BFS), "na mão", sem usar `cv.connectedComponents()`.
2. Compare a contagem de componentes de uma mesma imagem usando 4-conectividade e 8-conectividade.
3. Use `cv.connectedComponentsWithStats()` para colorir cada componente encontrado com uma cor diferente.
4. (Desafio) Filtre os componentes por área mínima, eliminando "ruído" (pequenos grupos de pixels isolados) antes de contar os objetos.

5.9 Resumo do capítulo

- Segmentação divide uma imagem em regiões — normalmente, objeto e fundo.
- 4-conectividade considera só vizinhos ortogonais; 8-conectividade também considera diagonais.
- Um algoritmo de rotulação em única passada falha em formas complexas — crescimento de região resolve isso.
- Crescimento de região pode ser implementado com fila (BFS), pilha (DFS) ou recursão.
- `cv.connectedComponentsWithStats()` rotula todos os componentes de uma imagem binária de uma vez, com estatísticas úteis (área, centroide, bounding box).

Capítulo 6 — Filtros Espaciais e Convolução I

Corresponde à Aula 6 (120 minutos).

6.1 Filtro espacial: uma janela que percorre a imagem

Um filtro espacial (ou filtro de vizinhança) calcula o valor de cada pixel de saída a partir dos valores de uma pequena JANELA de pixels ao redor dele, na imagem de entrada. Essa janela é chamada de kernel ou máscara de convolução, e desliza sobre toda a imagem, calculando um novo valor a cada posição.

Os modos "same" e "valid" descrevem o que fazer nas bordas da imagem, onde a janela ultrapassaria o domínio: no modo "same", a imagem de saída tem o MESMO tamanho da entrada (extrapolando ou ignorando os pixels fora do domínio de alguma forma); no modo "valid", a imagem de saída é menor, contendo apenas as posições onde a janela cabe inteiramente dentro da entrada.

6.2 Convolução

A convolução é a operação matemática por trás da maioria dos filtros espaciais lineares: para cada posição, multiplica-se cada valor da janela pelo peso correspondente do kernel, e soma-se tudo. É exatamente a mesma operação de "média aritmética ponderada" que veremos, mais adiante no curso, ser a base do casamento de template (Capítulo 9) e das redes neurais convolucionais (Capítulo 15).

```
fórmula da convolução
saída(x, y) =  $\sum_i \sum_j entrada(x+i, y+j) \times kernel(i, j)$ 
```

No OpenCV, `cv.filter2D()` aplica um kernel arbitrário (definido manualmente) a uma imagem — a base para implementar qualquer filtro linear personalizado.

6.3 Filtro da média (box filter)

O filtro de média (ou "média móvel") é o mais simples dos filtros de suavização: cada pixel de saída recebe a MÉDIA aritmética simples dos pixels dentro da janela. O efeito visual é um borramento (blur) — quanto maior a janela, mais forte o borramento. É útil para reduzir ruído, mas também borra bordas e detalhes finos.

```
conceito
cv.blur(origem, destino, new cv.Size(5, 5)); // janela 5x5
```

6.4 Filtro gaussiano

O filtro gaussiano também suaviza a imagem, mas em vez de dar o MESMO peso a todos os pixels da janela (como o filtro de média), ele dá pesos maiores aos pixels mais próximos do centro e pesos menores aos mais distantes, seguindo a curva da distribuição normal (gaussiana). O resultado costuma ser visualmente mais natural do que o filtro de média, preservando um pouco melhor os detalhes.

```
conceito
cv.GaussianBlur(origem, destino, new cv.Size(5, 5), 0);
// 0 = o desvio-padrão é calculado automaticamente a partir do tamanho da janela
```

Por que suavizar antes de outras operações?

Muitos algoritmos que veremos adiante no curso (detecção de bordas, no Capítulo 7; template matching, no Capítulo 9) são sensíveis a ruído. Aplicar um filtro gaussiano como pré-processamento é uma prática comum para tornar essas técnicas mais robustas.

6.5 Exemplo comentado: filtros de suavização

6.5.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');

  // filtro da média (box filter) 5x5
  let media = new cv.Mat();
  cv.blur(src, media, new cv.Size(5, 5));
  cv.imshow('canvasMedia', media);

  // filtro gaussiano 5x5
  let gauss = new cv.Mat();
  cv.GaussianBlur(src, gauss, new cv.Size(5, 5), 0);
  cv.imshow('canvasGauss', gauss);

  // kernel personalizado com filter2D (aqui, equivalente ao filtro de média)
  let kernel = cv.Mat.ones(3, 3, cv.CV_32FC1);
  for (let i = 0; i < 9; i++) kernel.data32F[i] = 1 / 9;
  let personalizado = new cv.Mat();
  cv.filter2D(src, personalizado, -1, kernel);
  cv.imshow('canvasPersonalizado', personalizado);

  media.delete(); gauss.delete(); kernel.delete(); personalizado.delete();
};
```

6.5.2 Versão Python — OpenCV-Python

```
aula06.py
import cv2
import numpy as np

img = cv2.imread("flor.jpg")

media = cv2.blur(img, (5, 5))
gauss = cv2.GaussianBlur(img, (5, 5), 0)

kernel = np.ones((3, 3), np.float32) / 9
personalizado = cv2.filter2D(img, -1, kernel)

cv2.imwrite("flor_media.jpg", media)
cv2.imwrite("flor_gauss.jpg", gauss)
```

6.5.3 Versão Node.js — opencv4nodejs

```

aula06.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg');

const media = img.blur(new cv.Size(5, 5));
const gauss = img.gaussianBlur(new cv.Size(5, 5), 0);

cv.imwrite('flor_media.jpg', media);
cv.imwrite('flor_gauss.jpg', gauss);

```

6.6 Erros comuns

- Usar uma janela par (ex.: 4x4) em vez de ímpar — a maioria dos filtros espaciais exige janelas de tamanho ímpar (3x3, 5x5, 7x7), para que exista um pixel central bem definido.
- Aumentar demais o tamanho da janela esperando "mais qualidade" — janelas grandes borram excessivamente a imagem e são mais lentas de calcular.
- Esquecer que `cv.blur()` e `cv.GaussianBlur()` reduzem ruído, mas também apagam detalhes finos e bordas.

6.7 Exercícios propostos

1. Aplique o filtro da média com janelas 3x3, 7x7 e 15x15 na mesma imagem e compare o grau de borramento.
2. Adicione ruído artificial ("sal e pimenta") a uma imagem e compare o resultado do filtro de média e do filtro gaussiano na remoção desse ruído.
3. Implemente, com `cv.filter2D()`, um kernel de "nitidez" (sharpen) que realça bordas em vez de suavizá-las.
4. (Desafio) Escreva manualmente (sem usar `cv.blur` ou `cv.GaussianBlur`) uma função de convolução 2D genérica, que recebe uma imagem e um kernel qualquer.

6.8 Resumo do capítulo

- Um filtro espacial calcula cada pixel de saída a partir de uma janela de vizinhança na imagem de entrada.
- Convolução multiplica cada valor da janela pelo peso do kernel e soma tudo — é a base dos filtros lineares.
- O filtro de média (`cv.blur`) dá peso igual a todos os pixels da janela; o filtro gaussiano (`cv.GaussianBlur`) dá mais peso ao centro.
- Ambos suavizam a imagem (reduzindo ruído), mas também borram bordas e detalhes.
- `cv.filter2D()` permite aplicar qualquer kernel personalizado a uma imagem.

Capítulo 7 — Filtros Espaciais II: Bordas e Contornos

Corresponde à Aula 7 (120 minutos).

7.1 Filtro mediano

O filtro mediano é diferente dos filtros de suavização vistos no Capítulo 6: em vez de calcular uma média (aritmética ou ponderada) dos pixels da janela, ele ORDENA os valores da janela e escolhe o valor do MEIO — a mediana. Isso o torna especialmente eficaz contra ruído do tipo "sal e pimenta" (pixels isolados totalmente brancos ou pretos), pois valores extremos e isolados são simplesmente descartados pela ordenação, em vez de "contaminarem" a média.

```
conceito
cv.medianBlur(origem, destino, 5); // janela 5x5, sempre ímpar
```

7.2 Gradiente e o filtro de Sobel

O gradiente de uma imagem mede o quanto a intensidade muda entre pixels vizinhos. Em regiões uniformes (mesma cor), o gradiente é próximo de zero; em bordas (transições bruscas de intensidade), o gradiente é alto. O filtro de Sobel aproxima o gradiente calculando duas derivadas: Gx (na direção horizontal) e Gy (na direção vertical). A magnitude do gradiente, calculada a partir de Gx e Gy, indica a força da borda em cada pixel.

```
conceito
cv.Sobel(cinza, gx, cv.CV_32F, 1, 0, 3); // derivada em x
cv.Sobel(cinza, gy, cv.CV_32F, 0, 1, 3); // derivada em y
cv.magnitude(gx, gy, magnitude); // combina gx e gy: sqrt(gx² + gy²)
```

Atenção ao tipo de dado

Ao calcular o gradiente com Sobel, use cv.CV_32F ou cv.CV_64F, não cv.CV_8U — o gradiente pode ser negativo, e um tipo sem sinal (8U) trunca esses valores negativos, corrompendo o resultado.

7.3 Filtro laplaciano

Enquanto o Sobel calcula a PRIMEIRA derivada (o gradiente) em uma direção específica, o filtro laplaciano calcula a SEGUNDA derivada, considerando todas as direções ao mesmo tempo com um único kernel. Isso o torna mais sensível a ruído do que o Sobel, mas também capaz de detectar bordas em qualquer orientação com uma única aplicação do filtro.

```
conceito
cv.Laplacian(cinza, destino, cv.CV_8U, 3); // kernel 3x3
```

7.4 Detetor de bordas de Canny

O algoritmo de Canny, desenvolvido por John Canny em 1986, é considerado o padrão da indústria para detecção de bordas. Ele combina várias etapas num único pipeline:

1. Suavização gaussiana, para reduzir a sensibilidade a ruído.
2. Cálculo do gradiente (magnitude e direção), semelhante ao Sobel.
3. Supressão de não-máximos: afina as bordas, mantendo só o pixel de maior gradiente em cada direção perpendicular à borda.
4. Limiarização com histerese: usa DOIS limiares (um baixo e um alto). Pixels acima do limiar alto são bordas "fortes"; pixels entre os dois limiares só são considerados borda se estiverem CONECTADOS a uma borda forte.

O resultado é um conjunto de bordas finas, contínuas e com muito menos ruído do que Sobel ou Laplaciano isolados.

```
conceito
cv.Canny(cinza, bordas,
        50, // limiar inferior
        150); // limiar superior
```

7.5 Exemplo comentado: mediano, Sobel, laplaciano e Canny

7.5.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  const img = document.querySelector('#minhaImagem');
  let src = cv.imread(img);
  let cinza = new cv.Mat();
  cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);
  cv.GaussianBlur(cinza, cinza, new cv.Size(5, 5), 0);

  let mediano = new cv.Mat();
  cv.medianBlur(cinza, mediano, 5);

  let laplace = new cv.Mat();
  cv.Laplacian(cinza, laplace, cv.CV_8U, 3);
  cv.imshow('canvasLaplace', laplace);

  let bordas = new cv.Mat();
  cv.Canny(cinza, bordas, 50, 150);
  cv.imshow('canvasCanny', bordas);

  mediano.delete(); laplace.delete(); bordas.delete();
};
```

7.5.2 Versão Python — OpenCV-Python

```
aula07.py
import cv2
```

```

img = cv2.imread("flor.jpg", cv2.IMREAD_GRAYSCALE)
blur = cv2.GaussianBlur(img, (5, 5), 0)

mediano = cv2.medianBlur(img, 5)
laplace = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
laplace = cv2.convertScaleAbs(laplace)

bordas = cv2.Canny(blur, 50, 150)

cv2.imwrite("flor_mediana.jpg", mediano)
cv2.imwrite("flor_laplace.jpg", laplace)
cv2.imwrite("flor_canny.jpg", bordas)

```

7.5.3 Versão Node.js — opencv4nodejs

```

aula07.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg', cv.IMREAD_GRAYSCALE);
const blur = img.gaussianBlur(new cv.Size(5, 5), 0);

const mediano = img.medianBlur(5);
const laplace = blur.laplacian(cv.CV_64F, { kSize: 3 });

const bordas = blur.canny(50, 150);

cv.imwrite('flor_mediana.jpg', mediano);
cv.imwrite('flor_canny.jpg', bordas);

```

7.6 Erros comuns

- Aplicar Canny sem suavizar antes — como ele é sensível a ruído, uma etapa de `cv.GaussianBlur()` antes evita bordas falsas.
- Escolher os limiares de Canny "no chute" — comece com uma razão de 1:2 ou 1:3 entre o limiar inferior e o superior, e ajuste observando o resultado.
- Usar `cv.CV_8U` no Sobel — trunca valores negativos do gradiente; prefira `cv.CV_32F` ou `cv.CV_64F`.

7.7 Exercícios propostos

1. Compare o filtro de média (Capítulo 6) com o filtro mediano na remoção de ruído "sal e pimenta".
2. Calcule o gradiente de uma imagem com Sobel e desenhe separadamente G_x , G_y e a magnitude combinada.
3. Aplique Canny com diferentes pares de limiares (ex.: 30/90, 50/150, 100/200) e observe como o resultado muda.
4. (Desafio) Combine Canny com inversão de cores (Capítulo 3) para simular um efeito de "desenho a lápis".

7.8 Resumo do capítulo

- O filtro mediano remove ruído "sal e pimenta" preservando bordas melhor do que a média ou o gaussiano.
- O filtro de Sobel calcula o gradiente (primeira derivada) nas direções X e Y.
- O filtro laplaciano calcula a segunda derivada, detectando bordas em qualquer direção com um único kernel.
- O algoritmo de Canny combina suavização, gradiente, supressão de não-máximos e histerese, sendo o detector de bordas mais usado na prática.

Capítulo 8 — Laboratório Integrador: Checkpoint Prático

Corresponde à Aula 8 (120 minutos).

8.1 Por que um checkpoint

Este capítulo não introduz teoria nova. Seu objetivo é consolidar, num único pipeline, as técnicas vistas nos Capítulos 1 a 7: espaços de cor, operações básicas, histograma e limiarização, componentes conexos e filtros espaciais. Combinar essas técnicas é justamente o que torna a visão computacional "aplicada" — poucos problemas reais são resolvidos com uma única operação isolada.

8.2 O projeto do checkpoint: detector de objetos por cor

O projeto guiado deste capítulo é um pipeline que recebe uma foto, identifica regiões de uma cor específica e desenha um retângulo ao redor de cada região encontrada, combinando cinco técnicas já estudadas:

1. Carregar a imagem e converter para o espaço de cor HSV (Capítulo 2).
2. Segmentar a cor de interesse com um limiar de intervalo — `cv.inRange()` (Capítulo 4).
3. Limpar o ruído da máscara resultante com um filtro (Capítulo 6).
4. Encontrar as regiões conectadas na máscara limpa — contornos ou componentes conexos (Capítulo 5).
5. Desenhando o resultado final sobre a imagem original (Capítulo 3).

8.3 A função `cv.inRange()`

`cv.inRange()` gera uma máscara binária a partir de um intervalo de valores: cada pixel cujo valor (em cada canal) está DENTRO do intervalo [baixo, alto] vira 255 na máscara; os demais viram 0. É a forma mais direta de segmentar uma cor específica em HSV, definindo os limites de matiz, saturação e valor que caracterizam a cor procurada.

```
conceito
cv.inRange(hsv, baixo, alto, mascara);
```

8.4 Exemplo comentado: pipeline completo

8.4.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');

  // 1. RGB -> HSV
  let hsv = new cv.Mat();
  cv.cvtColor(src, hsv, cv.COLOR_RGBA2RGB);
  cv.cvtColor(hsv, hsv, cv.COLOR_RGB2HSV);
```

```
// 2. Segmentar a cor de interesse
let baixo = new cv.Mat(hsv.rows, hsv.cols, hsv.type(), [130, 30, 100, 0]);
let alto = new cv.Mat(hsv.rows, hsv.cols, hsv.type(), [179, 255, 255, 255]);
let mascara = new cv.Mat();
cv.inRange(hsv, baixo, alto, mascara);

// 3. Limpar ruído
let elemento = cv.getStructuringElement(cv.MORPH_RECT, new cv.Size(3, 3));
cv.morphologyEx(mascara, mascara, cv.MORPH_OPEN, elemento);

// 4. Encontrar regiões
let contornos = new cv.MatVector();
cv.findContours(mascara, contornos, new cv.Mat(),
  cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE);

// 5. Desenhar resultado
for (let i = 0; i < contornos.size(); i++) {
  let rect = cv.boundingRect(contornos.get(i));
  cv.rectangle(src, new cv.Point(rect.x, rect.y),
    new cv.Point(rect.x + rect.width, rect.y + rect.height),
    [255, 0, 0, 255], 2);
}
cv.imshow('canvasSaida', src);
};
```

8.4.2 Versão Python — OpenCV-Python

```
aula08.py
import cv2
import numpy as np

img = cv2.imread("flor.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

baixo = np.array([130, 30, 100])
alto = np.array([179, 255, 255])
mascara = cv2.inRange(hsv, baixo, alto)

kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
mascara = cv2.morphologyEx(mascara, cv2.MORPH_OPEN, kernel)

contornos, _ = cv2.findContours(mascara, cv2.RETR_EXTERNAL,
                                cv2.CHAIN_APPROX_SIMPLE)

for c in contornos:
    x, y, w, h = cv2.boundingRect(c)
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 255), 2)

cv2.imwrite("flor_detectado.jpg", img)
```

8.4.3 Versão Node.js — opencv4nodejs

```
aula08.js
const cv = require('opencv4nodejs');
```

```
const img = cv.imread('flor.jpg');
const hsv = img.cvtColor(cv.COLOR_BGR2HSV);

const baixo = new cv.Vec3(130, 30, 100);
const alto = new cv.Vec3(179, 255, 255);
let mascara = hsv.inRange(baixo, alto);

const kernel = cv.getStructuringElement(cv.MORPH_RECT, new cv.Size(3, 3));
mascara = mascara.morphologyEx(kernel, cv.MORPH_OPEN);

const contornos = mascara.findContours(cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE);
contornos.forEach((c) => {
  const rect = c.boundingRect();
  img.drawRectangle(rect, new cv.Vec3(0, 0, 255), 2);
});

cv.imwrite('flor_detectado.jpg', img);
```

8.5 Erros comuns

- Máscara sempre vazia (tudo preto) — confira o número de canais: se a origem for RGBA, os Mats de baixo/alto precisam ter o mesmo número de canais para `cv.inRange()` funcionar corretamente.
- Muitos contornos pequenos e espúrios na máscara — aplique `cv.morphologyEx` (abertura, Capítulo 13) e filtre por `cv.contourArea()` antes de desenhar os resultados.
- Vazamento de memória no navegador — todo `cv.Mat()` criado precisa de `.delete()` ao final; acumular Mats sem liberar pode travar a aba.

8.6 Exercícios propostos

1. Troque a cor de interesse do pipeline, ajustando os valores de baixo/alto para detectar outra cor.
2. Mostre a contagem de objetos encontrados na tela, junto com o resultado desenhado.
3. Filtre por área mínima, ignorando contornos muito pequenos (ruído) com `cv.contourArea()`.
4. (Desafio) Desenhe também o centro de cada região com `cv.moments()` (o "centroide").

8.7 Resumo do capítulo

- Um pipeline real de visão computacional combina várias técnicas simples em sequência.
- Conversão de espaço de cor + limiarização (`cv.inRange`) já resolve boa parte da segmentação por cor.
- Contornos/componentes conexos transformam uma máscara de pixels em objetos identificáveis e mensuráveis.
- Cada `cv.Mat` criado no OpenCV.js deve ser liberado com `.delete()` para evitar vazamento de memória no navegador.

Capítulo 9 — Casamento de Template I

Corresponde à Aula 9 (120 minutos).

9.1 O que é casamento de template

Casamento de template (template matching), também chamado de casamento de modelo ou casamento de máscara, é uma técnica clássica de visão computacional usada para encontrar as ocorrências de um modelo pequeno Q (o "template") dentro de uma imagem maior A (a imagem de busca) — por exemplo, achar um logotipo dentro de uma foto, ou uma peça específica numa linha de produção industrial.

Template matching tradicionalmente usa correlação cruzada (essencialmente, a mesma convolução vista no Capítulo 6), e é uma das técnicas clássicas mais úteis: é computacionalmente leve, não exige treinar nenhum modelo, e — como veremos no Capítulo 15 — é, em certo sentido, o princípio matemático por trás da camada convolucional das redes neurais convolucionais.

9.2 Por que a correlação simples não basta

A ideia inicial mais simples é deslizar o modelo Q sobre a imagem A e, em cada posição, calcular o produto escalar entre os valores de Q e os valores de A dentro da janela — a mesma operação de um filtro linear (Capítulo 6). O problema é que essa correlação "crua" não é confiável: ela pode gerar falsos negativos e é sensível a brilho. Considere o vetor binário $a = [0,1,0,0,1,0,1,1,1,0,1,1,0]$ e o modelo $q = [0,1,1]$. Uma correlação direta produz picos ambíguos, sem distinguir claramente as duas ocorrências reais de "011" dentro de a .

9.3 Correção de média (dcReject) e o método CC

A solução é subtrair a MÉDIA do modelo Q de cada um dos seus valores antes de correlacionar — essa operação é chamada de correção de média. Ela torna a operação invariante a brilho: somar uma constante a toda a imagem A não altera mais o resultado da correlação.

Aplicando essa correção e correlacionando normalmente, obtemos o método CC (Cross-Correlation, ou correlação cruzada). A saída de CC é invariante a brilho, mas ainda é PROPORCIONAL AO CONTRASTE: uma instância de baixo contraste no template gera um pico mais baixo do que uma instância de alto contraste, mesmo que ambas representem o mesmo objeto.

```
conceito
cv.matchTemplate(A, Q, resultado, cv.TM_CCORR); // pré-processar Q com correção de
média antes
```


9.4 Coeficiente de correlação normalizada (NCC)

O método NCC (coeficiente de correlação normalizada, `TM_CCOEFF_NORMED` no OpenCV) vai além do CC: ele normaliza também pela energia (magnitude) da janela e do modelo. Matematicamente, calcula o cosseno do ângulo entre os dois vetores (o coeficiente de correlação de Pearson) — o resultado fica sempre entre -1 e +1, independentemente de brilho E de contraste.

```
conceito
cv.matchTemplate(A, Q, resultado, cv.TM_CCOEFF_NORMED);
// 1.0 = casamento perfeito | 0.0 = sem relação | -1.0 = padrão invertido
```

CC x NCC: um exemplo clássico

Imagine buscar a figura de um elefante de baixo contraste ("Dumbo") numa imagem cheia de outros personagens, incluindo um urso de alto contraste ("Baloo"). Usando CC, os pixels mais brilhantes do resultado NÃO indicam a localização do elefante, mas sim a do urso — porque CC é proporcional a contraste, e o urso tem contraste mais alto. Usando NCC, a localização correta do elefante é indicada, mesmo com seu baixo contraste, pois NCC é invariante tanto a brilho quanto a contraste.

9.5 A função `cv.matchTemplate()` e `cv.minMaxLoc()`

A função `cv.matchTemplate()` sempre trabalha no modo "valid" (Capítulo 6): a imagem de saída P é MENOR que a imagem de busca A, com dimensões $(\text{largura_A} - \text{largura_Q} + 1) \times (\text{altura_A} - \text{altura_Q} + 1)$. O pico de correlação (a melhor ocorrência do modelo em A) fica indicado no canto superior esquerdo do modelo no resultado P.

Para achar o MELHOR casamento — o pico de maior correlação — usamos `cv.minMaxLoc()`, que devolve o valor mínimo, o valor máximo e as posições onde eles ocorrem na matriz de resultado.

9.6 Exemplo comentado: casamento de template com $k=1$

9.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let cena = cv.imread('cena');
  let molde = cv.imread('molde');
  cv.cvtColor(cena, cena, cv.COLOR_RGBA2GRAY);
  cv.cvtColor(molde, molde, cv.COLOR_RGBA2GRAY);

  let resultado = new cv.Mat();
  cv.matchTemplate(cena, molde, resultado, cv.TM_CCOEFF_NORMED);

  // acha o pico de correlação (melhor casamento)
  let { maxVal, maxLoc } = cv.minMaxLoc(resultado);

  let ponto2 = new cv.Point(maxLoc.x + molde.cols, maxLoc.y + molde.rows);
  cv.rectangle(cena, maxLoc, ponto2, [0, 255, 0, 255], 2);
  cv.imshow('canvasSaida', cena);
}
```

```
console.log('Melhor casamento: ' + maxVal.toFixed(3));
};
```

9.6.2 Versão Python — OpenCV-Python

```
aula09.py
import cv2

cena = cv2.imread("cena.png", cv2.IMREAD_GRAYSCALE)
molde = cv2.imread("molde.png", cv2.IMREAD_GRAYSCALE)

resultado = cv2.matchTemplate(cena, molde, cv2.TM_CCOEFF_NORMED)
minVal, maxVal, minLoc, maxLoc = cv2.minMaxLoc(resultado)

h, w = molde.shape
cena_colorida = cv2.imread('cena.png')
cv2.rectangle(cena_colorida, maxLoc,
              (maxLoc[0] + w, maxLoc[1] + h), (0, 255, 0), 2)

print(f"Melhor casamento: {maxVal:.3f}")
cv2.imwrite("cena_detectada.png", cena_colorida)
```

9.6.3 Versão Node.js — opencv4nodejs

```
aula09.js
const cv = require('opencv4nodejs');

const cena = cv.imread('cena.png', cv.IMREAD_GRAYSCALE);
const molde = cv.imread('molde.png', cv.IMREAD_GRAYSCALE);

const resultado = cena.matchTemplate(molde, cv.TM_CCOEFF_NORMED);
const minMax = resultado.minMaxLoc();

const cenaColorida = cv.imread('cena.png');
const p2 = new cv.Point2(
  minMax.maxLoc.x + molde.cols, minMax.maxLoc.y + molde.rows);
cenaColorida.drawRectangle(minMax.maxLoc, p2, new cv.Vec3(0, 255, 0), 2);

console.log('Melhor casamento:', minMax.maxVal.toFixed(3));
cv.imwrite('cena_detectada.png', cenaColorida);
```

9.7 Erros comuns

- Comparar imagens de tamanhos ou tipos diferentes — `cv.matchTemplate` exige que a cena e o molde tenham o mesmo número de canais (ambos em escala de cinza, por exemplo).
- Achar que o resultado de `matchTemplate` tem o mesmo tamanho da cena — ele é sempre menor (modo "valid").
- Usar `TM_CCORR` quando o contraste do objeto pode variar — nesse caso, prefira `TM_CCOEFF_NORMED` (NCC).

9.8 Exercícios propostos

1. Recorte um pedaço pequeno (o "molde") de uma foto sua e busque-o na foto original com `TM_CCOEFF_NORMED`.
2. Compare o resultado usando `TM_CCORR` e `TM_CCOEFF_NORMED` — anote as diferenças.
3. Altere o brilho/contraste da foto de busca e veja como cada método reage.
4. (Desafio) Desenhe o valor de `maxVal` como texto sobre o retângulo do resultado.

9.9 Resumo do capítulo

- Casamento de template encontra um modelo Q dentro de uma imagem A por correlação.
- Correção de média (`dcReject`) torna a busca invariante a brilho.
- CC (`TM_CCORR`) é invariante a brilho, mas proporcional ao contraste.
- NCC (`TM_CCOEFF_NORMED`) é invariante a brilho E contraste — o método mais confiável na prática.
- `cv.minMaxLoc()` encontra a posição de maior correlação (o melhor casamento) na matriz de resultado.

Capítulo 10 — Casamento de Template II

Corresponde à Aula 10 (120 minutos).

10.1 Detectando múltiplas ocorrências

No Capítulo 9, usamos `cv.minMaxLoc()` para achar apenas o MELHOR casamento no mapa de correlação. Mas, se o modelo aparecer várias vezes na mesma imagem, precisamos de outra estratégia: percorrer TODA a matriz de resultado e coletar todas as posições cujo valor de correlação ultrapassa um limiar escolhido (por exemplo, $NCC \geq 0,7$).

```
conceito
let encontrados = [];
for (let y = 0; y < resultado.rows; y++) {
  for (let x = 0; x < resultado.cols; x++) {
    if (resultado.floatAt(y, x) >= 0.7) {
      encontrados.push({ x, y });
    }
  }
}
```

10.2 Evitando detecções duplicadas

Perto de cada casamento real, os pixels VIZINHOS também costumam ter correlação alta (o pico de correlação tem uma "vizinhança", não é um único ponto isolado). Sem cuidado, isso gera dezenas de retângulos sobrepostos para o mesmo objeto. A solução mais simples é filtrar, antes de aceitar uma nova detecção, se ela está muito próxima de uma detecção já aceita:

```
conceito
let perto = encontrados.some(p =>
  Math.abs(p.x - x) < 10 && Math.abs(p.y - y) < 10);
if (!perto) encontrados.push({ x, y }); // ignora vizinhos próximos
```

Técnicas mais sofisticadas de eliminação de duplicatas, como non-maximum suppression, são usadas em detectores de objetos mais avançados (Capítulo 15/16), mas o filtro de proximidade simples já resolve a maioria dos casos práticos.

10.3 Pixels "don't care"

Às vezes o modelo aparece parcialmente danificado, sujo ou parcialmente oculto na imagem de busca. Podemos marcar alguns pixels do modelo como "indiferentes" (don't care), para que eles não interfiram no cálculo da correlação. A partir da versão 4.4, o OpenCV permite passar uma MÁSCARA para `cv.matchTemplate()`, onde 255 indica "considerar o pixel" e 0 indica "ignorar (don't care)".

```
conceito
// mascara: 255 = considerar o pixel, 0 = ignorar (don't care)
cv.matchTemplate(cena, molde, resultado, cv.TM_CCORR_NORMED, mascara);
```

Um exemplo prático é a inspeção industrial: um risco ou sombra sobre parte de uma peça não deveria impedir a detecção do restante do padrão — usando uma máscara don't care sobre a área danificada, o casamento continua funcionando para o restante do padrão.

10.4 Limitações: escala e rotação

Template matching "puro" compara pixel a pixel, exatamente na mesma escala e orientação do modelo original. Se o objeto aparece girado ou em outro tamanho na cena, a correlação cai bastante — mesmo sendo visualmente "o mesmo" objeto para um observador humano.

Como lidar com escala e rotação

Uma estratégia comum (embora computacionalmente mais cara) é gerar várias versões do modelo, rotacionadas e/ou escaladas em diferentes ângulos e tamanhos, e buscar TODAS elas na cena, mantendo o melhor resultado. Técnicas mais robustas a essas variações (como descritores de características / feature matching) ficam fora do escopo desta disciplina, mas vale saber que existem.

10.5 Quando usar (e quando não usar) template matching

- Bom encaixe: objeto rígido, sempre no mesmo tamanho e ângulo (ex.: logotipo, código de barras); inspeção industrial com câmera fixa; poucos recursos computacionais (não exige GPU nem dataset de treino).
- Evite ou combine com outra técnica: objeto pode aparecer girado, em escala variável ou parcialmente oculto; classes muito variáveis (ex.: "gato" em geral, não um gato específico); cenas complexas com muita variação de iluminação e fundo.

10.6 Exemplo comentado: detectando todas as ocorrências

10.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let cena = cv.imread('cena'), molde = cv.imread('molde');
  cv.cvtColor(cena, cena, cv.COLOR_RGBA2GRAY);
  cv.cvtColor(molde, molde, cv.COLOR_RGBA2GRAY);

  let resultado = new cv.Mat();
  cv.matchTemplate(cena, molde, resultado, cv.TM_CCOEFF_NORMED);

  let limiar = 0.7, encontrados = [];
  for (let y = 0; y < resultado.rows; y++) {
    for (let x = 0; x < resultado.cols; x++) {
      if (resultado.floatAt(y, x) < limiar) continue;
      let perto = encontrados.some(p =>
        Math.abs(p.x - x) < 10 && Math.abs(p.y - y) < 10);
      if (!perto) encontrados.push({ x, y });
    }
  }
}
```

```

    encontrados.forEach(p => cv.rectangle(cena,
        new cv.Point(p.x, p.y),
        new cv.Point(p.x + molde.cols, p.y + molde.rows), [0, 255, 0, 255], 2));
    cv.imshow('canvasSaida', cena);
};

```

10.6.2 Versão Python — OpenCV-Python

```

aula10.py
import cv2
import numpy as np

cena = cv2.imread("cena.png", cv2.IMREAD_GRAYSCALE)
molde = cv2.imread("molde.png", cv2.IMREAD_GRAYSCALE)
h, w = molde.shape

resultado = cv2.matchTemplate(cena, molde, cv2.TM_CCOEFF_NORMED)
limiar = 0.7
ys, xs = np.where(resultado >= limiar)

cena_colorida = cv2.imread('cena.png')
encontrados = []
for (x, y) in zip(xs, ys):
    if not any(abs(x - ex) < 10 and abs(y - ey) < 10 for ex, ey in encontrados):
        encontrados.append((x, y))
        cv2.rectangle(cena_colorida, (x, y), (x + w, y + h), (0, 255, 0), 2)

print(f"{len(encontrados)} ocorrências encontradas")
cv2.imwrite("cena multipla.png", cena_colorida)

```

10.6.3 Versão Node.js — opencv4nodejs

```

aula10.js
const cv = require('opencv4nodejs');

const cena = cv.imread('cena.png', cv.IMREAD_GRAYSCALE);
const molde = cv.imread('molde.png', cv.IMREAD_GRAYSCALE);

const resultado = cena.matchTemplate(molde, cv.TM_CCOEFF_NORMED);
const limiar = 0.7;
let encontrados = [];

for (let y = 0; y < resultado.rows; y++) {
    for (let x = 0; x < resultado.cols; x++) {
        if (resultado.at(y, x) < limiar) continue;
        const perto = encontrados.some(p =>
            Math.abs(p.x - x) < 10 && Math.abs(p.y - y) < 10);
        if (!perto) encontrados.push({ x, y });
    }
}
console.log(encontrados.length, 'ocorrências encontradas');

```

10.7 Erros comuns

- Um único objeto gera várias caixas sobrepostas — sempre filtre por proximidade (ou use non-maximum suppression) antes de desenhar os resultados.
- Limiar fixo funciona bem numa imagem mas falha em outra — ajuste o limiar observando o histograma do mapa de correlação; não existe valor mágico universal.
- Esperar robustez a rotação/escala "de graça" — template matching puro não tem essa robustez nativamente.

10.8 Exercícios propostos

1. Monte uma cena com 3 ou mais cópias do mesmo objeto e detecte todas com um único limiar.
2. Implemente o filtro de proximidade para eliminar caixas duplicadas.
3. Teste o mesmo detector numa versão rotacionada da cena — confirme que ele falha, como esperado.
4. (Desafio) Gere uma máscara "don't care" cobrindo parte do modelo e compare os resultados com e sem a máscara.

10.9 Resumo do capítulo

- Para achar todas as ocorrências de um modelo, percorremos o mapa de correlação inteiro com um limiar.
- É preciso filtrar picos vizinhos para não desenhar caixas duplicadas sobre o mesmo objeto.
- Máscaras "don't care" (a partir do OpenCV 4.4) permitem ignorar partes danificadas ou irrelevantes do modelo.
- Template matching puro não é robusto a mudanças de escala e rotação — é importante conhecer os limites da técnica antes de aplicá-la.

Capítulo 11 — Transformações Geométricas I

Corresponde à Aula 11 (120 minutos).

11.1 O problema da reamostragem

Mudar a resolução de uma imagem (ampliar ou reduzir) exige calcular a cor de pixels que não existiam na imagem original. Podemos imaginar a imagem como uma função contínua no domínio 2D, amostrada em certos pontos (os pixels originais). Reamostrar significa estimar o valor dessa função em NOVOS pontos, conhecendo apenas os valores nos pontos originais — esse processo de estimativa é chamado de interpolação.

11.2 Interpolação vizinho mais próximo

A solução mais simples é a interpolação vizinho mais próximo (nearest neighbor): cada pixel da imagem de saída recebe a cor do pixel espacialmente mais próximo na imagem de entrada.

```
conceito
saida(l, c) = entrada( arredonda(l / fatorL), arredonda(c / fatorC) );
```

O fato de vários pixels de saída poderem receber a mesma cor gera um efeito indesejável de blocos de cor uniforme (o efeito "escadinha"), visível principalmente ao ampliar bastante a imagem.

11.3 Interpolação bilinear

A interpolação bilinear elimina esse efeito de blocos calculando a MÉDIA ARITMÉTICA PONDERADA dos 4 pixels de entrada que circundam o pixel de saída, com pesos proporcionais à distância (quanto mais perto, maior o peso).

```
conceito
peso1 = (1-dc) (1-dl)    peso2 = dc(1-dl)
peso3 = (1-dc)dl         peso4 = dc·dl

saida = peso1*a1 + peso2*a2 + peso3*a3 + peso4*a4;
```

onde dl e dc são as distâncias fracionárias (entre 0 e 1) do ponto de saída até os pixels de entrada mais próximos, nas direções de linha e coluna respectivamente.

11.4 Interpolações bicúbica e Lanczos

As interpolações bicúbica e Lanczos levam em consideração vizinhanças ainda maiores — 4×4 e 8×8 pixels, respectivamente — produzindo resultados progressivamente mais suaves, ao custo de mais processamento. O OpenCV oferece a função pronta `cv.resize()` com os seguintes métodos de interpolação:

- `INTER_NEAREST` — vizinho mais próximo (mais rápida, menor qualidade).
- `INTER_LINEAR` — bilinear (padrão do `cv.resize()`, bom equilíbrio custo/qualidade).

- INTER_CUBIC — bicúbica, sobre vizinhança 4×4.
- INTER_LANCZOS4 — Lanczos, sobre vizinhança 8×8 (mais lenta, maior qualidade).
- INTER_AREA — reamostragem por relação de área, ideal para REDUZIR imagens (ver seção seguinte).

11.5 O problema do aliasing ao reduzir imagens

As interpolações vistas acima funcionam bem para AMPLIAR imagens. Mas, ao REDUZIR uma imagem que contém padrões finos (texturas, listras, grades), amostrar poucos pixels pode criar padrões FALSOS que não existiam na imagem original — esse efeito é chamado de aliasing ou padrão de Moiré.

Uma explicação intuitiva: ao reduzir a imagem, uma região pode acabar amostrando majoritariamente um valor (por exemplo, a cor de um tijolo), enquanto uma região vizinha amostra majoritariamente outro valor (a cor da argamassa) — criando faixas alternadas que não representam fielmente a textura original.

Como evitar aliasing

A prática recomendada é aplicar um filtro passa-baixa (tipicamente um filtro gaussiano, Capítulo 6) antes de reduzir a resolução — em vez de amostrar um único pixel, a MÉDIA de uma região é amostrada, evitando o aliasing. O OpenCV oferece a interpolação cv.INTER_AREA, projetada especificamente para reduzir imagens sem gerar esse efeito, calculando a média ponderada por área de sobreposição entre os pixels de entrada e saída.

11.6 Exemplo comentado: cv.resize() com diferentes interpolações

11.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');
  let dst = new cv.Mat();
  let tamanho = new cv.Size(src.cols * 2, src.rows * 2);

  // ampliar com bicúbica (melhor qualidade)
  cv.resize(src, dst, tamanho, 0, 0, cv.INTER_CUBIC);
  cv.imshow('canvasAmpliado', dst);

  // reduzir sem aliasing, usando INTER_AREA
  let reduzida = new cv.Mat();
  let tamanhoMenor = new cv.Size(src.cols * 0.4, src.rows * 0.4);
  cv.resize(src, reduzida, tamanhoMenor, 0, 0, cv.INTER_AREA);
  cv.imshow('canvasReduzido', reduzida);
};
```

11.6.2 Versão Python — OpenCV-Python

```
aula11.py
import cv2
```

```
img = cv2.imread("flor.jpg")

ampliado = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
reduzido = cv2.resize(img, None, fx=0.4, fy=0.4, interpolation=cv2.INTER_AREA)

cv2.imwrite("flor_ampliada.jpg", ampliado)
cv2.imwrite("flor_reduzida.jpg", reduzido)
```

11.6.3 Versão Node.js — opencv4nodejs

```
aula11.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg');

const ampliado = img.resize(img.rows * 2, img.cols * 2, {
  interpolation: cv.INTER_CUBIC,
});
const reduzido = img.resize(img.rows * 0.4, img.cols * 0.4, {
  interpolation: cv.INTER_AREA,
});

cv.imwrite('flor_ampliada.jpg', ampliado);
cv.imwrite('flor_reduzida.jpg', reduzido);
```

11.7 Erros comuns

- Usar `INTER_NEAREST` para reduzir imagens com texturas finas — gera aliasing/moiré; prefira `cv.INTER_AREA` para reduzir.
- Confundir `cv.Size(largura, altura)` com a ordem (linhas, colunas) — `cv.resize` espera `Size` na ordem (x, y), o oposto da ordem usada para indexar pixels em uma `Mat`.
- Ampliar demais uma imagem pequena esperando nitidez — interpolação não "inventa" detalhes que não existiam, apenas suaviza a transição entre pixels.

11.8 Exercícios propostos

1. Amplie uma foto sua 3x usando `INTER_NEAREST` e `INTER_CUBIC` e compare visualmente.
2. Reduza uma imagem com padrões finos (uma foto de tecido ou tela) usando `INTER_NEAREST` e depois `INTER_AREA` — compare o aliasing.
3. Meça o tempo de execução de cada interpolação e compare a relação custo/benefício.
4. (Desafio) Implemente manualmente (sem `cv.resize`) a interpolação bilinear para ampliar uma imagem pequena.

11.9 Resumo do capítulo

- Reamostrar uma imagem exige interpolar valores de pixels que não existiam originalmente.
- Vizinho mais próximo é rápido mas gera blocos; bilinear, bicúbica e Lanczos são progressivamente mais suaves e mais caras.

- Reduzir imagens com padrões finos pode gerar aliasing/Moiré — use `cv.INTER_AREA` para evitar.
- `cv.resize()` é a função pronta do OpenCV para mudar a resolução de uma imagem, com vários métodos de interpolação disponíveis.

Capítulo 12 — Transformações Geométricas II

Corresponde à Aula 12 (120 minutos).

12.1 Translação

Translação move cada pixel (x, y) de uma imagem para uma nova posição $(x + tx, y + ty)$, deslocando a imagem inteira. Pode ser descrita como uma matriz 2×3 — o mesmo formato usado pelo OpenCV para representar qualquer transformação afim (translação, rotação, escala e cisalhamento combinados):

```
conceito
M = [ 1  0  tx ]
    [ 0  1  ty ]
// tx, ty = deslocamento em x e y
```

12.2 A matriz de rotação 2D

Para rotacionar uma imagem em torno da ORIGEM do sistema de coordenadas por um ângulo θ , cada pixel é multiplicado pela matriz de rotação:

```
conceito
c = cos(θ)    s = sen(θ)

M = [ c  -s ]
    [ s   c ]
```

Na prática, para desenhar a imagem de SAÍDA, precisamos da transformação INVERSA: para cada pixel de saída, calculamos de qual pixel da imagem de ENTRADA ele deveria vir (evitando "buracos" na imagem de saída que ocorreriam se percorrêssemos a entrada e calculássemos onde cada pixel iria parar).

12.3 Três problemas práticos da rotação "na mão"

Implementar rotação manualmente, aplicando a fórmula acima diretamente, esbarra em três problemas:

1. Sistema de coordenadas: o OpenCV acessa pixels no sistema (linha, coluna), não no sistema cartesiano (x, y) usado na fórmula.
2. Centro de rotação: a fórmula gira a imagem em torno do canto superior esquerdo $(0,0)$ — na prática, quase sempre queremos girar em torno do CENTRO da imagem.
3. Pixels fora do domínio: a rotação pode levar a coordenadas que não existem na imagem de entrada, exigindo uma decisão sobre o que fazer nesses casos (preencher com uma cor constante, replicar a borda, etc.).

Por resolver esses três problemas de uma vez, o OpenCV oferece funções prontas em vez de exigir essa implementação manual em cada projeto.

12.4 cv.getRotationMatrix2D() e cv.warpAffine()

cv.getRotationMatrix2D() monta a matriz afim 2×3 já considerando o centro de rotação escolhido, e até um fator de escala opcional (permitindo combinar rotação e zoom numa única chamada).

cv.warpAffine() aplica essa matriz (ou qualquer outra matriz afim) a uma imagem, cuidando automaticamente dos pixels fora do domínio.

```
conceito
let centro = new cv.Point(src.cols / 2, src.rows / 2);
let M = cv.getRotationMatrix2D(centro, 30, 1.0);
    // ponto central, ângulo em graus, fator de escala

let dst = new cv.Mat();
cv.warpAffine(src, dst, M, new cv.Size(src.cols, src.rows));
```

Sentido da rotação

Como consequência do sistema de coordenadas usado pelo OpenCV (eixo y crescendo para BAIXO, e não para cima como na convenção matemática tradicional), um ângulo POSITIVO passado para getRotationMatrix2D gira a imagem no sentido HORÁRIO.

12.5 Transformação afim genérica: cv.getAffineTransform()

A transformação afim é mais geral do que rotação e translação: é qualquer transformação que preserve linhas retas e o paralelismo entre elas (o que inclui também o cisalhamento, ou "shear"). O OpenCV calcula a matriz afim a partir de apenas 3 pares de pontos correspondentes (origem → destino) com cv.getAffineTransform().

```
conceito
let origem = cv.matFromArray(3, 1, cv.CV_32FC2, [0,0, w-1,0, 0,h-1]);
let destino = cv.matFromArray(3, 1, cv.CV_32FC2, [0,0, w-1,0, w*0.25,h-1]);

let M = cv.getAffineTransform(origem, destino);
cv.warpAffine(src, dst, M, new cv.Size(w, h));
```

12.6 Exemplo comentado: translação e rotação

12.6.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
    let src = cv.imread('minhaImagem');

    // 1. Translação: matriz 2x3 manual
    let M1 = cv.matFromArray(2, 3, cv.CV_64F, [1, 0, 60, 0, 1, 30]);
    let transladada = new cv.Mat();
    cv.warpAffine(src, transladada, M1, new cv.Size(src.cols, src.rows));
    cv.imshow('canvasTranslacao', transladada);

    // 2. Rotação: getRotationMatrix2D + warpAffine
    let centro = new cv.Point(src.cols / 2, src.rows / 2);
```

```

let M2 = cv.getRotationMatrix2D(centro, 30, 1.0);
let rotacionada = new cv.Mat();
cv.warpAffine(src, rotacionada, M2, new cv.Size(src.cols, src.rows));
cv.imshow('canvasRotacao', rotacionada);
};

```

12.6.2 Versão Python — OpenCV-Python

```

aula12.py
import cv2
import numpy as np

img = cv2.imread("flor.jpg")
h, w = img.shape[:2]

# translação
M1 = np.float32([[1, 0, 60], [0, 1, 30]])
transladada = cv2.warpAffine(img, M1, (w, h))

# rotação em torno do centro
M2 = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)
rotacionada = cv2.warpAffine(img, M2, (w, h))

cv2.imwrite("flor_transladada.jpg", transladada)
cv2.imwrite("flor_rotacionada.jpg", rotacionada)

```

12.6.3 Versão Node.js — opencv4nodejs

```

aula12.js
const cv = require('opencv4nodejs');

const img = cv.imread('flor.jpg');
const { rows, cols } = img;

// translação
const M1 = new cv.Mat([[1, 0, 60], [0, 1, 30]], cv.CV_64F);
const transladada = img.warpAffine(M1);

// rotação em torno do centro
const centro = new cv.Point2(cols / 2, rows / 2);
const M2 = cv.getRotationMatrix2D(centro, 30, 1.0);
const rotacionada = img.warpAffine(M2);

cv.imwrite('flor_transladada.jpg', transladada);
cv.imwrite('flor_rotacionada.jpg', rotacionada);

```

12.7 Erros comuns

- Rotacionar em torno de (0,0) em vez do centro — sempre passe o centro desejado para `cv.getRotationMatrix2D`, não presume que seja automático.
- Esquecer o tamanho de saída em `cv.warpAffine()` — se for diferente do tamanho de entrada, a imagem resultante pode ser cortada.

- Confundir ângulo positivo com sentido anti-horário — no sistema de coordenadas do OpenCV, ângulos positivos giram no sentido horário.

12.8 Exercícios propostos

1. Translade uma imagem controlando tx e ty com dois sliders HTML.
2. Rotacione uma imagem em torno do seu centro com um slider de 0° a 360°.
3. Combine rotação e escala numa única chamada de `cv.getRotationMatrix2D()`.
4. (Desafio) Implemente o "desafio Dumbo/Baloo" do Capítulo 9, agora buscando o modelo em várias rotações diferentes.

12.9 Resumo do capítulo

- Translação e rotação podem ser expressas como uma única matriz de transformação afim 2×3.
- `cv.getRotationMatrix2D()` monta a matriz de rotação já considerando o centro escolhido e um fator de escala opcional.
- `cv.warpAffine()` aplica qualquer matriz afim a uma imagem, cuidando de pixels fora do domínio.
- `cv.getAffineTransform()` calcula a matriz a partir de 3 pares de pontos — útil para cisalhamento e correções de perspectiva simples.

Capítulo 13 — Morfologia Matemática

Corresponde à Aula 13 (120 minutos).

13.1 O que é morfologia matemática

Morfologia matemática é um conjunto de operações não lineares aplicadas tipicamente sobre imagens BINÁRIAS (máscaras preto e branco), usadas para limpar ruído, separar objetos conectados por engano, unir fragmentos de um mesmo objeto e extrair estruturas geométricas. Diferente dos filtros lineares (Capítulo 6), que ponderam e somam valores, as operações morfológicas comparam a vizinhança de cada pixel com um padrão de referência chamado elemento estruturante.

13.2 O elemento estruturante

O elemento estruturante é uma pequena matriz binária (por exemplo 3×3 ou 5×5) que define a vizinhança e o formato considerados em cada operação — um retângulo, uma cruz, ou uma elipse são formatos comuns. O OpenCV oferece `cv.getStructuringElement()` para gerar os formatos mais usados:

```
conceito
let elemento = cv.getStructuringElement(cv.MORPH_RECT, new cv.Size(3, 3));
// outros formatos: cv.MORPH_CROSS, cv.MORPH_ELLIPSE
```

13.3 Erosão

A erosão "encolhe" as regiões brancas (o primeiro plano) de uma imagem binária: um pixel só permanece branco no resultado se TODOS os pixels sob o elemento estruturante, centrado nele, também forem brancos na imagem original. Isso remove pequenos ruídos isolados e afina os objetos.

```
conceito
cv.erode(src, dst, elemento);
```

13.4 Dilatação

A dilatação faz o oposto: "expande" as regiões brancas. Um pixel se torna branco no resultado se PELO MENOS UM pixel sob o elemento estruturante, centrado nele, for branco na imagem original. Isso preenche pequenos buracos e conecta fragmentos próximos de um mesmo objeto.

```
conceito
cv.dilate(src, dst, elemento);
```

O formato do elemento importa

Um elemento estruturante em cruz erode/dilata de forma diferente de um elemento retangular ou elíptico nas diagonais — o formato escolhido deve refletir o tipo de estrutura que se deseja preservar ou remover.

13.5 Abertura (opening)

A abertura é a composição de uma EROSAÇÃO seguida de uma DILATAÇÃO, com o mesmo elemento estruturante. O efeito prático é remover pequenos ruídos e saliências finas do primeiro plano, preservando o tamanho geral dos objetos maiores (a erosão remove o ruído, e a dilatação seguinte "devolve" o tamanho original aos objetos que sobreviveram).

```
conceito
cv.morphologyEx(src, dst, cv.MORPH_OPEN, elemento);
```

13.6 Fechamento (closing)

O fechamento é o inverso: uma DILATAÇÃO seguida de uma EROSAÇÃO. O efeito prático é preencher pequenos buracos e conectar fragmentos próximos de um mesmo objeto, sem alterar significativamente seu tamanho externo.

```
conceito
cv.morphologyEx(src, dst, cv.MORPH_CLOSE, elemento);
```

Abertura x fechamento — quando usar cada uma

Use ABERTURA quando o ruído está no FUNDO (pequenos pontos brancos espalhados sobre um fundo preto que deveriam ser removidos). Use FECHAMENTO quando o ruído está no OBJETO (pequenos buracos pretos dentro de uma região branca que deveriam ser preenchidos). Frequentemente as duas são aplicadas em sequência para limpar uma máscara real.

13.7 Exemplo comentado: limpando uma máscara ruidosa

13.7.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let mascara = cv.imread('minhaMascara');
  cv.cvtColor(mascara, mascara, cv.COLOR_RGBA2GRAY);

  let elemento = cv.getStructuringElement(cv.MORPH_RECT, new cv.Size(3, 3));

  // abertura: remove ruído pequeno no fundo
  let aberta = new cv.Mat();
  cv.morphologyEx(mascara, aberta, cv.MORPH_OPEN, elemento);

  // fechamento: preenche buracos pequenos no objeto
  let limpa = new cv.Mat();
  cv.morphologyEx(aberta, limpa, cv.MORPH_CLOSE, elemento);

  cv.imshow('canvasSaida', limpa);
};
```

13.7.2 Versão Python — OpenCV-Python

```
aula13.py
```

```
import cv2

mascara = cv2.imread("mascara_ruidosa.png", cv2.IMREAD_GRAYSCALE)
elemento = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))

aberta = cv2.morphologyEx(mascara, cv2.MORPH_OPEN, elemento)
limpa = cv2.morphologyEx(aberta, cv2.MORPH_CLOSE, elemento)

cv2.imwrite("mascara_limpa.png", limpa)
```

13.7.3 Versão Node.js — opencv4nodejs

```
aula13.js
const cv = require('opencv4nodejs');

const mascara = cv.imread('mascara_ruidosa.png', cv.IMREAD_GRAYSCALE);
const elemento = cv.getStructuringElement(cv.MORPH_RECT, new cv.Size(3, 3));

const aberta = mascara.morphologyEx(elemento, cv.MORPH_OPEN);
const limpa = aberta.morphologyEx(elemento, cv.MORPH_CLOSE);

cv.imwrite('mascara_limpa.png', limpa);
```

13.8 Erros comuns

- Aplicar abertura quando o problema são buracos no objeto (deveria ser fechamento) — identifique se o ruído está no fundo ou dentro do objeto antes de escolher a operação.
- Elemento estruturante grande demais — pode apagar detalhes legítimos junto com o ruído; comece sempre com elementos pequenos (3×3) e aumente apenas se necessário.
- Aplicar morfologia em imagem colorida ou em tons de cinza contínuos — as operações clássicas de erosão/dilatação binária são pensadas para máscaras binárias (0 ou 255).

13.9 Exercícios propostos

1. Gere uma máscara binária ruidosa (por exemplo, com `cv.inRange` do Capítulo 8) e aplique erosão e dilatação separadamente — compare os efeitos.
2. Aplique abertura e fechamento na mesma máscara e compare com o resultado de cada operação isolada.
3. Troque o elemento estruturante de retangular para elíptico e observe as diferenças nas bordas dos objetos.
4. (Desafio) Combine abertura seguida de fechamento (ambas) para limpar uma máscara com ruído tanto no fundo quanto dentro dos objetos.

13.10 Resumo do capítulo

- Operações morfológicas comparam a vizinhança de cada pixel a um elemento estruturante, tipicamente sobre imagens binárias.
- Erosão encolhe regiões brancas; dilatação as expande.

- Abertura (erosão + dilatação) remove ruído pequeno no fundo, preservando o tamanho dos objetos.
- Fechamento (dilatação + erosão) preenche buracos pequenos dentro dos objetos.
- `cv.getStructuringElement()` e `cv.morphologyEx()` são as funções centrais do módulo de morfologia no OpenCV.

Capítulo 14 — Introdução ao Aprendizado de Máquina

Corresponde à Aula 14 (120 minutos).

14.1 De técnicas clássicas para aprendizado de máquina

Nos capítulos anteriores, toda técnica de visão computacional foi construída a partir de REGRAS explícitas definidas por nós: um limiar de cor, um elemento estruturante, uma fórmula de correlação. Aprendizado de máquina inverte essa lógica: em vez de programar as regras, fornecemos EXEMPLOS (dados) e um algoritmo aprende os padrões automaticamente a partir deles.

14.2 Três paradigmas de aprendizado

- **Aprendizado supervisionado:** o algoritmo aprende a partir de exemplos rotulados (entrada + resposta correta conhecida) — por exemplo, milhares de imagens de dígitos manuscritos, cada uma já identificada com o número correto. É o paradigma mais usado em visão computacional aplicada.
- **Aprendizado não supervisionado:** o algoritmo recebe apenas dados SEM rótulos e busca padrões ou agrupamentos (clusters) por conta própria — por exemplo, agrupar pixels de uma imagem por similaridade de cor sem informar previamente quantos grupos existem.
- **Aprendizado por reforço:** um agente aprende por tentativa e erro, recebendo recompensas ou punições por suas ações num ambiente — mais comum em robótica e jogos do que em visão computacional pura, mas cada vez mais combinado com ela (por exemplo, um robô que aprende a pegar objetos usando visão como entrada).

14.3 k-NN (k-vizinhos mais próximos)

k-NN é um dos algoritmos de aprendizado supervisionado mais simples de entender: para classificar uma nova amostra, o algoritmo observa as k amostras de treino MAIS PRÓXIMAS dela (segundo alguma medida de distância, geralmente euclidiana) e atribui a classe mais frequente entre elas.

```
conceito
distancia(A, B) = sqrt( (Ax-Bx)2 + (Ay-By)2 + ... )
// classifica a nova amostra pela classe majoritária entre os k vizinhos mais próximos
```

Escolhendo k

Um k pequeno (ex.: k=1) torna o modelo sensível a ruído (uma amostra de treino atípica pode decidir sozinha a classificação). Um k grande demais suaviza demais as fronteiras entre classes e pode ignorar padrões locais importantes. Não existe valor universal de k — o ideal é testado empiricamente para cada problema.

14.4 Árvores de decisão

Uma árvore de decisão classifica uma amostra fazendo uma sequência de perguntas binárias sobre seus atributos (por exemplo: "o valor médio de vermelho é maior que 150?"), até chegar a uma folha que

indica a classe prevista. É um modelo fácil de interpretar (basta seguir o caminho de perguntas) e serve de base para modelos mais avançados, como florestas aleatórias (random forests).

14.5 Vetor de características (features)

Antes de aplicar qualquer algoritmo clássico de aprendizado de máquina a uma imagem, é comum reduzi-la a um VETOR DE CARACTERÍSTICAS (features) — um conjunto de números que resume aspectos relevantes da imagem (cor média, histograma, textura, formato do contorno), em vez de usar todos os pixels diretamente. Essa etapa, chamada de extração de características, é uma das diferenças centrais entre aprendizado de máquina clássico e redes neurais profundas (Capítulo 15), que aprendem as próprias características automaticamente a partir dos pixels brutos.

14.6 cv.KNearest no OpenCV

O módulo ml do OpenCV oferece cv.KNearest, uma implementação pronta do algoritmo k-NN, treinada a partir de uma matriz de amostras (cada linha é uma amostra, cada coluna é uma característica) e um vetor de rótulos correspondentes.

14.7 Exemplo comentado: classificando pontos 2D com k-NN

14.7.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  // amostras de treino: pontos 2D (x, y) e seus rótulos (0 ou 1)
  let amostras = cv.matFromArray(6, 2, cv.CV_32F,
    [1,1, 2,1, 1,2, 8,8, 9,8, 8,9]);
  let rotulos = cv.matFromArray(6, 1, cv.CV_32F, [0,0,0, 1,1,1]);

  let knn = new cv.KNearest();
  knn.train(amostras, cv.ml.ROW_SAMPLE, rotulos);

  // classificar uma nova amostra
  let novaAmostra = cv.matFromArray(1, 2, cv.CV_32F, [1.5, 1.5]);
  let resultado = new cv.Mat();
  let classe = knn.findNearest(novaAmostra, 3, resultado);
  console.log('Classe prevista:', classe);
};
```

14.7.2 Versão Python — OpenCV-Python

```
aula14.py
import cv2
import numpy as np

amostras = np.array(
  [[1, 1], [2, 1], [1, 2], [8, 8], [9, 8], [8, 9]], dtype=np.float32)
rotulos = np.array([0, 0, 0, 1, 1, 1], dtype=np.float32)

knn = cv2.ml.KNearest_create()
```

```
knn.train(amostras, cv2.ml.ROW_SAMPLE, rotulos)

nova_amostra = np.array([[1.5, 1.5]], dtype=np.float32)
_, resultado, vizinhos, distancias = knn.findNearest(nova_amostra, k=3)
print(f"Classe prevista: {resultado[0][0]}")
```

14.7.3 Versão Node.js — opencv4nodejs

```
aula14.js
const cv = require('opencv4nodejs');

const amostras = new cv.Mat(
  [[1, 1], [2, 1], [1, 2], [8, 8], [9, 8], [8, 9]], cv.CV_32F);
const rotulos = new cv.Mat([0], [0], [0], [1], [1], [1]], cv.CV_32F);

const knn = new cv.KNearest();
knn.train(amostras, rotulos);

const novaAmostra = new cv.Mat([[1.5, 1.5]], cv.CV_32F);
const resultado = knn.findNearest(novaAmostra, 3);
console.log('Classe prevista:', resultado.result);
```

14.8 Erros comuns

- Usar pixels brutos como características sem normalizar ou reduzir dimensionalidade — geralmente produz resultados fracos; prefira extrair características relevantes (cor média, histograma, forma).
- Escolher k par em k-NN com apenas duas classes — pode gerar empates na votação; prefira valores ímpares de k.
- Avaliar o modelo apenas com os dados de treino — sempre reserve uma parte dos dados para teste, nunca vista durante o treinamento, para medir a real capacidade de generalização.

14.9 Exercícios propostos

1. Monte um conjunto de treino com pelo menos 3 classes de pontos 2D e classifique novas amostras com cv.KNearest.
2. Varie o valor de k (1, 3, 5, 7) e observe como a classificação de um mesmo ponto de teste muda.
3. Extraia a cor média (R, G, B) de várias imagens de duas categorias diferentes (por exemplo, frutas verdes x frutas maduras) e use como vetor de características para k-NN.
4. (Desafio) Implemente manualmente (sem cv.KNearest) o cálculo de distância euclidiana e a votação por maioria entre k vizinhos.

14.10 Resumo do capítulo

- Aprendizado de máquina aprende padrões a partir de exemplos, em vez de seguir regras explícitas programadas manualmente.
- Os três paradigmas principais são supervisionado, não supervisionado e por reforço.

- k-NN classifica uma amostra pela classe majoritária entre seus k vizinhos mais próximos — simples de entender e implementar.
- Modelos clássicos de aprendizado de máquina em imagens costumam depender de um vetor de características extraído manualmente, diferentemente das redes neurais profundas (Capítulo 15).

Capítulo 15 — Redes Neurais e Redes Neurais Convolucionais (CNNs)

Corresponde à Aula 15 (120 minutos).

15.1 O neurônio artificial

Um neurônio artificial recebe várias entradas numéricas, multiplica cada uma por um peso (aprendido durante o treinamento), soma tudo com um termo de viés (bias) e passa o resultado por uma função de ativação, que introduz não linearidade ao modelo:

```
conceito
z = w1*x1 + w2*x2 + ... + wn*xn + bias;
saida = ativacao(z);
```

Sem a função de ativação, empilhar vários neurônios em camadas equivaleria matematicamente a uma única transformação linear — a não linearidade é o que permite à rede aprender padrões complexos.

15.2 Funções de ativação comuns

- Sigmoid: comprime qualquer valor para o intervalo (0, 1) — útil na camada de saída para problemas de classificação binária.
- ReLU (Rectified Linear Unit): retorna o valor de entrada se for positivo, e 0 caso contrário — a mais usada em camadas intermediárias por ser simples e eficiente de treinar.
- Softmax: transforma um vetor de valores em uma distribuição de probabilidades que somam 1 — usada na camada de saída para classificação com múltiplas classes.

15.3 Perceptron multicamadas (MLP)

Um perceptron multicamadas organiza neurônios em camadas sequenciais: uma camada de entrada, uma ou mais camadas ocultas (hidden layers) e uma camada de saída. Cada neurônio de uma camada se conecta a TODOS os neurônios da camada seguinte (camada densamente conectada, ou fully connected). O treinamento ajusta os pesos de todas essas conexões usando o algoritmo de retropropagação (backpropagation), comparando a saída da rede com a resposta correta esperada.

Por que MLPs não são ideais para imagens

Uma imagem de 224×224 pixels em RGB tem mais de 150 mil valores. Conectar cada pixel a cada neurônio da primeira camada oculta de um MLP exigiria milhões de pesos, tornando o treinamento caro e propenso a overfitting. Além disso, um MLP tradicional ignora a estrutura espacial da imagem — não sabe, a priori, que pixels vizinhos estão relacionados. As redes convolucionais resolvem os dois problemas.

15.4 Convolução como filtro aprendido

O Capítulo 6 apresentou a convolução com kernels definidos manualmente (Sobel, Gaussiano, Laplaciano). Uma rede neural convolucional (CNN) usa exatamente a mesma operação matemática de convolução — mas, em vez de definirmos os valores do kernel manualmente, eles são PARÂMETROS APRENDIDOS durante o treinamento. A rede descobre sozinha, a partir dos dados, quais filtros são úteis para a tarefa (detectar bordas, texturas, formas, e em camadas mais profundas, padrões cada vez mais abstratos e complexos).

```
conceito
// analogia: um Sobel do Capítulo 6 é um kernel FIXO, escolhido por nós
// uma camada convolucional de uma CNN é um conjunto de kernels APRENDIDOS
```

15.5 Pooling

Camadas de pooling (geralmente max pooling) reduzem a resolução espacial do mapa de características entre camadas convolucionais, mantendo apenas o valor máximo (ou a média) de cada pequena região. Isso reduz o custo computacional das camadas seguintes e torna a rede mais robusta a pequenos deslocamentos do objeto na imagem.

15.6 Arquitetura típica de uma CNN

Uma CNN típica de classificação de imagens alterna blocos de convolução + ativação (ReLU) + pooling, extraindo características cada vez mais abstratas, até finalmente "achatar" o resultado (flatten) e passá-lo por uma ou mais camadas densas (como num MLP) que produzem a classificação final, geralmente com uma camada softmax de saída.

15.7 Usando modelos já treinados: cv.dnn

Treinar uma CNN do zero exige grandes datasets rotulados e poder computacional considerável (normalmente GPUs). Na prática, a maioria das aplicações de visão computacional usa modelos JÁ TREINADOS por terceiros (em datasets massivos como ImageNet ou COCO) e apenas os executa (inferência) sobre novas imagens. O módulo cv.dnn do OpenCV permite carregar modelos treinados em frameworks populares (ONNX, TensorFlow, Caffe) e rodar inferência diretamente.

```
conceito
let net = cv.readNetFromONNX(bufferDoModelo);
let blob = cv.blobFromImage(imagem, 1/255, new cv.Size(224, 224));
net.setInput(blob);
let saida = net.forward();
```

15.8 Exemplo comentado: inferência com cv.dnn

15.8.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js

```
js/script.js
```

```

cv['onRuntimeInitialized'] = function () {
  fetch('modelo.onnx')
    .then((r) => r.arrayBuffer())
    .then((buffer) => {
      let net = cv.readNetFromONNX(new Uint8Array(buffer));

      let src = cv.imread('minhaImagem');
      let blob = cv.blobFromImage(src, 1 / 255.0, new cv.Size(224, 224),
        new cv.Scalar(0, 0, 0), true, false);

      net.setInput(blob);
      let saida = net.forward();
      console.log('Saída da rede:', saida.data32F);
    });
};

```

15.8.2 Versão Python — OpenCV-Python

```

aula15.py
import cv2

net = cv2.dnn.readNetFromONNX("modelo.onnx")

img = cv2.imread("flor.jpg")
blob = cv2.dnn.blobFromImage(img, 1/255.0, (224, 224),
                             swapRB=True, crop=False)

net.setInput(blob)
saida = net.forward()
print("Saída da rede:", saida)

```

15.8.3 Versão Node.js — opencv4nodejs

```

aula15.js
const cv = require('opencv4nodejs');

const net = cv.readNetFromONNX('modelo.onnx');

const img = cv.imread('flor.jpg');
const blob = cv.blobFromImage(img, 1 / 255.0, new cv.Size(224, 224),
  new cv.Vec3(0, 0, 0), true, false);

net.setInput(blob);
const saida = net.forward();
console.log('Saída da rede:', saida.getDataAsArray());

```

15.9 Erros comuns

- Esquecer de normalizar os pixels de entrada (geralmente dividindo por 255) antes de passar pela rede — a maioria dos modelos espera valores entre 0 e 1, não entre 0 e 255.
- Não redimensionar a imagem para o tamanho de entrada esperado pelo modelo — `cv.blobFromImage()` faz isso automaticamente, mas o tamanho passado precisa bater com o que o modelo foi treinado para receber.

- Inverter a ordem dos canais (RGB x BGR) — verifique se o parâmetro `swapRB` do `blobFromImage` corresponde ao que o modelo espera.

15.10 Exercícios propostos

1. Desenhe à mão (papel ou ferramenta de desenho) a arquitetura de uma CNN simples com 2 blocos de convolução+pooling e 1 camada densa final.
2. Pesquise e explique com suas palavras a diferença entre ReLU e Sigmoid como função de ativação.
3. Baixe um modelo ONNX pré-treinado simples (por exemplo, um classificador de dígitos MNIST) e rode inferência sobre uma imagem de teste usando `cv.dnn`.
4. (Desafio) Compare o tempo de inferência do mesmo modelo ONNX nas três linguagens (Web, Python, Node.js).

15.11 Resumo do capítulo

- Um neurônio artificial combina entradas ponderadas com uma função de ativação não linear.
- MLPs conectam todos os neurônios entre camadas, o que é ineficiente e espacialmente "cego" para imagens.
- CNNs usam convolução com kernels APRENDIDOS (em vez de definidos manualmente) para extrair características de imagens de forma eficiente.
- Pooling reduz a resolução espacial entre camadas convolucionais, tornando a rede mais eficiente e robusta a pequenos deslocamentos.
- `cv.dnn` permite carregar e rodar modelos já treinados (ONNX, TensorFlow, Caffe) sem precisar treinar uma rede do zero.

Capítulo 16 — Aplicações de IA em Visão Computacional

Corresponde à Aula 16 (120 minutos).

16.1 Detecção de rostos com `cv.CascadeClassifier`

Antes das CNNs se popularizarem, a técnica dominante para detecção de rostos em tempo real era o classificador em cascata de Viola-Jones, ainda hoje disponível no OpenCV como `cv.CascadeClassifier`. Ele usa um conjunto de classificadores simples (baseados em características tipo Haar) organizados em cascata: cada estágio descarta rapidamente regiões que claramente NÃO são rostos, deixando apenas as regiões promissoras para os estágios seguintes, mais custosos — o que torna a técnica muito rápida mesmo sem GPU.

```
conceito
let classificador = new cv.CascadeClassifier();
classificador.load('haarcascade_frontalface_default.xml');

let rostos = new cv.RectVector();
classificador.detectMultiScale(cinza, rostos);
```

Haar cascade x CNN

O classificador em cascata é mais rápido e leve, mas menos preciso e mais sensível a variações de pose, iluminação e oclusão do que um detector baseado em CNN moderno (Capítulo 15). Continua sendo uma boa escolha para aplicações com poucos recursos computacionais ou que rodam diretamente no navegador.

16.2 Reconhecimento com `cv.dnn`

Para tarefas mais exigentes — reconhecer objetos específicos, classificar cenas, detectar múltiplas classes de objetos com caixas delimitadoras (object detection) — o módulo `cv.dnn` (Capítulo 15) permite carregar modelos pré-treinados mais sofisticados, como variantes de YOLO, MobileNet-SSD ou ResNet, e aplicá-los diretamente sobre imagens ou vídeo.

16.3 OCR: reconhecimento óptico de caracteres

OCR (Optical Character Recognition) converte texto presente numa imagem em texto digital editável/pesquisável. No navegador, a biblioteca Tesseract.js oferece OCR pronto, baseado no motor Tesseract (desenvolvido originalmente pela HP e hoje mantido pelo Google), sem precisar de nenhum servidor: todo o processamento roda localmente, no próprio JavaScript.

```
conceito
Tesseract.recognize(imagem, 'por')
  .then(({ data: { text } }) => console.log(text));
```

16.4 Por que pré-processar antes do OCR

Motores de OCR como o Tesseract são treinados majoritariamente sobre texto limpo e bem contrastado (digitalizações de documentos, por exemplo). Fotos reais de texto — placas, embalagens, anotações à mão, fotos tiradas em ângulo ou com iluminação ruim — costumam ter ruído, baixo contraste ou fundo desigual, o que derruba drasticamente a taxa de acerto do OCR.

As técnicas clássicas de processamento de imagem estudadas ao longo desta disciplina são exatamente o que resolve esse problema: um pipeline típico de pré-processamento para OCR usa escala de cinza (Capítulo 2), suavização para reduzir ruído — como filtro de mediana (Capítulo 7) — e binarização adaptativa com limiar de Otsu (Capítulo 4), antes de enviar a imagem ao motor de OCR.

Exemplo verificado

Um texto fotografado com ruído "sal e pimenta" pronunciado, quando enviado diretamente ao Tesseract, produz um resultado cheio de erros de reconhecimento. O mesmo texto, após um pipeline de suavização gaussiana seguida de limiarização Otsu, é reconhecido corretamente quase na íntegra pelo mesmo motor de OCR — demonstrando, na prática, que técnicas clássicas de visão computacional continuam essenciais mesmo em pipelines modernos apoiados por IA.

16.5 Exemplo comentado: pipeline OpenCV + OCR

16.5.1 Versão Web — HTML5 + CSS + JavaScript + OpenCV.js + Tesseract.js

```
js/script.js
cv['onRuntimeInitialized'] = function () {
  let src = cv.imread('minhaImagem');
  let cinza = new cv.Mat();
  cv.cvtColor(src, cinza, cv.COLOR_RGBA2GRAY);

  // suavizar ruído
  let suave = new cv.Mat();
  cv.GaussianBlur(cinza, suave, new cv.Size(5, 5), 0);

  // binarizar com Otsu
  let bin = new cv.Mat();
  cv.threshold(suave, bin, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU);

  cv.imshow('canvasPreprocessado', bin);

  // OCR sobre a imagem pré-processada
  let canvas = document.getElementById('canvasPreprocessado');
  Tesseract.recognize(canvas, 'por').then(({ data: { text } }) => {
    document.querySelector('#resultadoOCR').textContent = text;
  });
};
```

16.5.2 Versão Python — OpenCV-Python + pytesseract

```
aula16.py
```

```
import cv2
import pytesseract

img = cv2.imread("texto_ruidoso.jpg", cv2.IMREAD_GRAYSCALE)
suave = cv2.GaussianBlur(img, (5, 5), 0)
_, bin_img = cv2.threshold(suave, 0, 255,
                           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

texto = pytesseract.image_to_string(bin_img, lang='por')
print(texto)
```

16.5.3 Versão Node.js — opencv4nodejs + node-tesseract-ocr

```
aula16.js
const cv = require('opencv4nodejs');
const tesseract = require('node-tesseract-ocr');

const img = cv.imread('texto_ruidoso.jpg', cv.IMREAD_GRAYSCALE);
const suave = img.gaussianBlur(new cv.Size(5, 5), 0);
const bin = suave.threshold(0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU);

cv.imwrite('preprocessado.png', bin);

tesseract.recognize('preprocessado.png', { lang: 'por' })
  .then((texto) => console.log(texto));
```

16.6 Erros comuns

- Rodar OCR direto na foto original sem nenhum pré-processamento — resultado tipicamente ruim em fotos com ruído ou baixo contraste.
- Esquecer de carregar o arquivo .xml do cascade antes de chamar detectMultiScale() — o classificador precisa ser explicitamente carregado com .load().
- Escolher o idioma errado no Tesseract (por exemplo, 'eng' para texto em português) — reduz drasticamente a taxa de acerto.

16.7 Exercícios propostos

1. Use cv.CascadeClassifier para detectar rostos numa foto com várias pessoas e desenhe um retângulo ao redor de cada um.
2. Fotografe (ou simule) um texto com ruído e rode Tesseract.js direto, sem pré-processamento — anote os erros de reconhecimento.
3. Aplique o pipeline de pré-processamento (suavização + Otsu) na mesma imagem e rode o OCR novamente — compare os resultados.
4. (Desafio) Combine detecção de rosto com um classificador simples (k-NN do Capítulo 14) para tentar estimar se a pessoa está sorrindo, usando alguma característica simples da região da boca.

16.8 Resumo do capítulo

- cv.CascadeClassifier oferece detecção de rostos rápida e leve, baseada em características Haar em cascata.
- cv.dnn permite usar modelos de reconhecimento mais sofisticados, treinados em datasets massivos.
- Tesseract.js roda OCR completo no navegador, sem depender de servidor.
- Pré-processar imagens com técnicas clássicas (suavização, binarização Otsu) melhora significativamente a taxa de acerto de motores de OCR.
- Visão computacional aplicada, na prática, combina técnicas clássicas e modernas de IA — raramente uma substitui totalmente a outra.

Capítulo 17 — Projeto Final

Corresponde à Aula 17 (120 minutos).

17.1 Objetivo do projeto final

O projeto final é o momento de reunir, num único produto, as técnicas estudadas ao longo de toda a disciplina: fundamentos de processamento de imagem (Capítulos 1-4), segmentação e filtros (Capítulos 5-8), casamento de template e transformações geométricas (Capítulos 9-13) e, quando fizer sentido, técnicas de aprendizado de máquina e IA (Capítulos 14-16). Não é esperado um projeto de grande escala — o mais importante é aplicar corretamente os conceitos estudados a um problema real, com um pipeline coerente do início ao fim.

17.2 Temas sugeridos

- Detector de objetos por cor com contagem (evolução do Capítulo 8: além de detectar, classificar por tamanho/forma e contar quantos objetos foram encontrados).
- Leitor de placas ou códigos simples (combinando limiarização, morfologia e OCR — Capítulos 4, 13 e 16).
- Verificador de qualidade/inspeção industrial simples (comparação por template matching entre uma peça padrão e peças de teste — Capítulos 9-10).
- Editor de imagem interativo no navegador (combinando várias operações estudadas — brilho/contraste, filtros, transformações geométricas — controladas por sliders HTML).
- Classificador simples de imagens (usando k-NN sobre um vetor de características extraído manualmente — Capítulo 14).

O aluno também pode propor um tema próprio, desde que aprovado previamente pelo professor e que aplique de forma clara pelo menos três técnicas distintas estudadas na disciplina.

17.3 Requisitos mínimos

1. Pipeline funcional, executável no navegador (HTML5 + CSS + JavaScript + OpenCV.js), com upload de imagem ou acesso à webcam.
2. Aplicação de pelo menos três técnicas distintas estudadas na disciplina, combinadas de forma coerente (não apenas justapostas).
3. Interface simples que permita ao usuário interagir com o pipeline (sliders, botões, ou controles equivalentes) e visualizar o resultado.
4. Código organizado e comentado, explicando o papel de cada etapa do pipeline.
5. Breve documentação (README ou seção equivalente) explicando o problema resolvido, as técnicas usadas e como executar o projeto.

17.4 Rubrica de avaliação sugerida

- Correção técnica (40%): as técnicas de visão computacional foram aplicadas corretamente, com resultados visualmente coerentes com o esperado.
- Integração do pipeline (25%): as etapas se conectam de forma lógica, cada uma preparando adequadamente os dados para a próxima.
- Interface e usabilidade (15%): a aplicação é fácil de usar e comunica claramente o que está acontecendo em cada etapa.
- Documentação e clareza do código (10%): comentários e documentação permitem que outra pessoa entenda e reproduza o projeto.
- Originalidade e esforço (10%): o projeto vai além do mínimo exigido, demonstrando domínio genuíno do conteúdo.

17.5 Estudo de caso: do problema ao pipeline

Um exemplo de raciocínio para estruturar um projeto: suponha o tema "contador de moedas numa mesa". O pipeline poderia ser estruturado assim: (1) converter para escala de cinza e aplicar suavização para reduzir ruído de textura da mesa (Capítulos 2 e 6-7); (2) binarizar com limiar de Otsu, já que moedas geralmente contrastam bem com a mesa (Capítulo 4); (3) limpar a máscara resultante com abertura morfológica, removendo pequenos ruídos e separando moedas encostadas (Capítulo 13); (4) encontrar os componentes conexos (moedas) e filtrar por área mínima, ignorando ruído residual (Capítulo 5); (5) desenhar um círculo ao redor de cada moeda encontrada e exibir a contagem total na tela (Capítulo 3).

Comece pelo pipeline, não pelo código

Antes de escrever qualquer linha de código, esboce no papel (ou num diagrama simples) a sequência de etapas do seu pipeline, como no exemplo acima. Isso evita retrabalho e ajuda a identificar, com antecedência, qual capítulo da apostila revisar para cada etapa.

17.6 Cronograma sugerido de trabalho

1. Escolha do tema e esboço do pipeline no papel (revisar os capítulos relevantes da apostila).
2. Implementação da primeira versão funcional do pipeline, mesmo que simplificada.
3. Refinamento: ajuste de parâmetros (limiares, elementos estruturantes, k) testando em diferentes imagens de entrada.
4. Construção da interface de interação (sliders, botões, upload de imagem/webcam).
5. Documentação final e revisão do código antes da entrega.

17.7 Encerramento da disciplina

Ao longo desta disciplina, percorremos o caminho desde os fundamentos de um pixel até pipelines completos combinando técnicas clássicas de processamento de imagem com aprendizado de máquina e redes neurais convolucionais. A visão computacional aplicada é, acima de tudo, uma disciplina de PIPELINE: raramente uma única técnica resolve um problema real sozinha — a habilidade de combinar as ferramentas certas, na ordem certa, para o problema certo, é o que diferencia uma solução funcional de uma solução frágil. Esse é o principal aprendizado que esperamos que o projeto final consolide.

17.8 Resumo do capítulo

- O projeto final integra, num pipeline coerente, pelo menos três técnicas estudadas ao longo da disciplina.
- A rubrica avalia correção técnica, integração do pipeline, interface, documentação e originalidade.
- Esboçar o pipeline antes de codificar economiza tempo e retrabalho.
- Visão computacional aplicada é, fundamentalmente, a arte de combinar técnicas simples em soluções robustas para problemas reais.